

LAPORAN TUGAS *PROJECT*
MATA KULIAH BASIS DATA
SISTEM BASIS DATA RESERVASI HOTEL



Oleh Kelompok G:

1. Nasywa Putri Maitsa (K1D024043)
2. Nayla Edenine Qohar (K1D024044)
3. Abdurrahman Yusuf (K1D024058)
4. Bertha Misella Silalahi (K1D024068)

KEMENTERIAN PENDIDIKAN TINGGI, SAINS DAN TEKNOLOGI
UNIVERSITAS JENDERAL SOEDIRMAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI S1 STATISTIKA
PURWOKERTO
2026

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Industri perhotelan merupakan salah satu sektor jasa yang terus berkembang seiring meningkatnya mobilitas masyarakat dan kebutuhan akomodasi. Dalam operasional hotel, pengelolaan data reservasi kamar, data tamu, pembayaran, check-in, check-out, serta penggunaan fasilitas menjadi aspek yang sangat penting untuk menjamin pelayanan yang efektif dan efisien.

Pengelolaan data secara manual berpotensi menimbulkan berbagai permasalahan, seperti kesalahan pencatatan reservasi, duplikasi data tamu, kesulitan dalam memantau ketersediaan kamar, hingga keterlambatan dalam penyusunan laporan operasional hotel. Oleh karena itu, diperlukan suatu sistem basis data yang mampu mengintegrasikan seluruh data operasional hotel secara terstruktur dan konsisten.

Sistem Reservasi Hotel merupakan solusi yang dapat digunakan untuk mengelola informasi kamar, tamu, reservasi, pembayaran, penggunaan fasilitas, serta proses check-in dan check-out secara terpusat. Dengan penerapan basis data relasional, data dapat disimpan secara terorganisasi, meminimalkan redundansi, menjaga integritas data, serta memudahkan proses pengolahan informasi.

Berdasarkan permasalahan tersebut, dilakukan perancangan dan implementasi Sistem Basis Data Reservasi Hotel sebagai penerapan konsep basis data relasional yang telah dipelajari selama perkuliahan.

1.2 Rumusan Masalah

1. Bagaimana merancang sistem basis data yang mampu mengelola proses reservasi hotel secara terintegrasi?
2. Bagaimana membangun model data yang sesuai dengan kebutuhan operasional hotel?
3. Bagaimana menerapkan normalisasi hingga bentuk normal ketiga (3NF)?
4. Bagaimana mengimplementasikan basis data menggunakan DBMS relasional?
5. Bagaimana membuat query, view, function, procedure, dan trigger untuk mendukung operasional hotel?

1.3 Tujuan

1. Merancang sistem basis data reservasi hotel yang terstruktur dan terintegrasi.
2. Membuat Entity Relationship Diagram (ERD) dan skema relasional.
3. Menerapkan proses normalisasi hingga Third Normal Form (3NF).
4. Mengimplementasikan basis data menggunakan SQL.
5. Membuat query dan objek basis data lanjutan untuk mendukung pengelolaan hotel.

1.4 Manfaat

a. Bagi Hotel

Pembuatan Sistem Basis Data Reservasi Hotel ini diharapkan dapat memberikan manfaat bagi berbagai pihak. Bagi pihak hotel, sistem ini dapat membantu dalam mengelola data operasional secara lebih efektif dan terstruktur, mulai dari pengelolaan data tamu, reservasi kamar, pembayaran, hingga penggunaan fasilitas hotel. Dengan adanya sistem basis data yang terintegrasi, proses pencatatan dan pencarian informasi dapat dilakukan dengan lebih cepat sehingga dapat meningkatkan kualitas pelayanan kepada pelanggan serta mengurangi risiko kesalahan pengolahan data.

b. Bagi Pengguna

Bagi pengguna atau tamu hotel, sistem ini memberikan kemudahan dalam proses reservasi dan memperoleh informasi terkait kamar maupun fasilitas yang tersedia. Selain itu, data transaksi dan riwayat pemesanan dapat terdokumentasi dengan baik sehingga meningkatkan kenyamanan dan kepercayaan pelanggan terhadap layanan hotel.

c. Bagi Mahasiswa

Bagi mahasiswa sebagai pengembang sistem, project ini menjadi sarana untuk menerapkan konsep-konsep basis data yang telah dipelajari selama perkuliahan, seperti analisis kebutuhan, perancangan Entity Relationship Diagram (ERD), normalisasi data, implementasi basis data relasional, serta pembuatan query dan objek basis data lanjutan. Melalui project ini, mahasiswa juga dapat mengembangkan kemampuan pemecahan masalah, kerja sama tim, dan dokumentasi teknis yang akan bermanfaat dalam dunia kerja maupun pengembangan sistem informasi di masa mendatang.

BAB 2

ANALISIS KEBUTUHAN

2.1 Deskripsi Studi Kasus

Hotel merupakan entitas bisnis yang menyediakan jasa akomodasi berbayar bagi tamu. Sistem reservasi hotel adalah inti dari operasional front-office yang mengelola keseluruhan siklus menginap tamu, mulai dari pemesanan kamar, check-in, penggunaan fasilitas, check-out, hingga pembayaran. Seluruh proses ini melibatkan berbagai aktor dan data yang saling berkaitan.

Hotel Reservation System adalah project Basis Data dengan studi kasus Sistem Reservasi Hotel. Project ini dirancang untuk mengelola data tamu, pegawai, tipe kamar, kamar, fasilitas, reservasi, detail reservasi, pembayaran, check-in, check-out, dan log aktivitas menggunakan MySQL 8.0.

Project ini berfokus pada perancangan database relasional yang lengkap, mulai dari analisis kebutuhan, normalisasi, ERD, DDL, DML, query pengujian, view, stored procedure, function, trigger, hingga database dump.

Ruang Lingkup Sistem

Ruang lingkup project meliputi:

1. Pengelolaan data tamu.
2. Pengelolaan data pegawai.
3. Pengelolaan tipe kamar dan kamar hotel.
4. Pengelolaan detail reservasi dan detail kamar yang di pesan.
5. Pencegahan double booking kamar berdasarkan periode tanggal reservasi yang bertabrakan.
6. Validasi kamar berstatus `Perawatan` agar tidak dapat dipesan, Perubahan status kamar otomatis dari `Tersedia`, `Dipesan`, `Terisi`, hingga kembali `Tersedia`.
7. Pengelolaan pembayaran.
8. Pencatatan proses check-in dan check-out.
9. Pengelolaan fasilitas kamar
10. Pencatatan log aktivitas pegawai.
11. Penyediaan view untuk laporan detail reservasi dan pembayaran.
12. Penyediaan stored procedure, function, dan trigger sebagai implementasi lanjutan basis data.

Sistem ini berfokus pada perancangan dan implementasi database, bukan pada pengembangan antarmuka aplikasi. Sistem ini tidak mencakup manajemen SDM hotel secara menyeluruh (penggajian, jadwal shift), manajemen inventori dapur/restoran, maupun sistem akuntansi lanjutan.

2.2 Identifikasi Aktor/Pengguna

Aktor	Peran
Tamu	Melakukan registrasi identitas diri, memilih tipe kamar, melakukan pembayaran deposit/lunas, dan menginap.
Resepsionis	Mengelola pemetaan kamar kosong, memverifikasi data check-in tamu datang, memproses kepulangan check-out, dan memperbarui status kamar fisik.
Kasir	Mencatat transaksi pembayaran dari tamu, memilih metode pembayaran, memvalidasi sisa tagihan, dan mengeluarkan invoice.
Supervisor	Memantau kinerja operasional hotel melalui laporan audit trail, okupansi kamar harian, laporan bulanan keuangan, dan performa produktivitas staf.

2.3 Kebutuhan Fungsional

No	Kebutuhan	Deskripsi
1	Kelola Data Tamu	Menyimpan identitas tamu hotel.
2	Kelola Data Pegawai	Menyimpan data pegawai yang menangani operasional hotel.
3	Kelola Tipe Kamar	Menyimpan kategori kamar, kapasitas, dan harga.
4	Kelola Kamar	Menyimpan nomor kamar, lantai, tipe, dan status kamar.
5	Kelola Fasilitas	Menyimpan daftar fasilitas hotel.
6	Kelola Fasilitas	Mencatat fasilitas pada setiap kamar.

	Kamar	
7	Kelola Reservasi	Mencatat transaksi reservasi tamu.
8	Kelola Detail Reservasi	Mencatat kamar yang dipesan dalam reservasi.
9	Kelola Pembayaran	Mencatat pembayaran reservasi.
10	Kelola Check-in	Mencatat kedatangan aktual tamu.
11	Kelola Check-out	Mencatat kepulangan aktual tamu dan biaya tambahan.
12	Kelola Laporan	Menyediakan laporan reservasi dan pembayaran.
13	Kelola Log Aktivitas	Mencatat aktivitas penting dalam sistem.

2.4 Kebutuhan Non-Fungsional

No	Kebutuhan	Deskripsi
1	Integritas Data	Data dijaga menggunakan PK, FK, UNIQUE, CHECK, dan DEFAULT.
2	Konsistensi Data	Nilai status dan metode pembayaran dibatasi menggunakan ENUM.
3	Ketersediaan Data	Basis data dapat digunakan untuk operasional dan pengujian query.
4	Kemudahan Pelaporan	View dan query disediakan untuk membantu penyusunan laporan.
5	Skalabilitas	Struktur tabel memungkinkan penambahan data hotel di masa mendatang.

2.5 Business Rules

1. Setiap tamu harus terdaftar dalam sistem sebelum melakukan reservasi.
2. Nomor identitas dan email tamu harus bersifat unik.
3. Setiap reservasi harus terkait dengan satu tamu.

4. Setiap reservasi harus dicatat atau ditangani oleh seorang pegawai.
5. Tanggal check-out harus lebih besar dari tanggal check-in.
6. Satu reservasi dapat mencakup satu atau lebih kamar.
7. Satu kamar hanya boleh digunakan oleh satu reservasi pada periode yang sama.
8. Setiap kamar harus memiliki satu tipe kamar.
9. Nomor kamar harus unik.
10. Status kamar harus selalu diperbarui sesuai kondisi terkini.
11. Subtotal detail reservasi dihitung dari harga per malam dikalikan jumlah malam menginap.
12. Reservasi dapat memiliki satu atau lebih transaksi pembayaran.
13. Jumlah pembayaran yang diterima tidak boleh bernilai negatif.
14. Status pembayaran harus menunjukkan kondisi pembayaran yang sebenarnya.
15. Check-in hanya dapat dilakukan untuk reservasi yang valid dan aktif.
16. Setiap reservasi hanya dapat memiliki satu data check-in.
17. Check-out hanya dapat dilakukan setelah proses check-in tercatat.
18. Setiap reservasi hanya dapat memiliki satu data check-out.
19. Data check-in dan check-out harus dicatat oleh pegawai yang bertugas.
20. Biaya tambahan saat check-out tidak boleh bernilai negatif.
21. Satu kamar dapat memiliki banyak fasilitas dan satu fasilitas dapat tersedia pada banyak kamar.
22. Nama fasilitas harus unik.
23. Setiap aktivitas penting yang dilakukan pegawai harus dicatat dalam log aktivitas.
24. Email pegawai harus unik.
25. Data yang sudah digunakan dalam transaksi reservasi tidak boleh dihapus secara langsung.

BAB 3

PERANCANGAN BASIS DATA

3.1 Identifikasi Entitas

Database Sistem Reservasi Hotel terdiri atas 12 entitas utama sebagai berikut.

No	Entitas	Fungsi
1	tamu	Menyimpan data identitas tamu.
2	pegawai	Menyimpan data pegawai hotel.
3	tipe_kamar	Menyimpan kategori kamar dan harga.
4	fasilitas	Menyimpan data fasilitas.
5	kamar	Menyimpan data kamar fisik hotel.
6	reservasi	Menyimpan transaksi reservasi.
7	detail_reservasi	Menyimpan rincian kamar dalam reservasi.
8	pembayaran	Menyimpan data pembayaran reservasi.
9	checkin	Menyimpan data check-in.
10	checkout	Menyimpan data check-out.
11	kamar_fasilitas	Menyimpan relasi antara kamar dan fasilitas.
12	log_aktivitas	Menyimpan aktivitas sistem.

3.2 Logical Design (Skema Relasional)

Pemetaan ERD ke skema relasional menghasilkan representasi berikut (PK = Primary Key, FK = Foreign Key, UQ = Unique):

```
tamu (id_tamu PK, nama_tamu, no_identitas UQ, jenis_kelamin, no_telepon, email UQ, alamat, is_active, created_at, updated_at)

pegawai (id_pegawai PK, nama_pegawai, jabatan, no_telepon, email UQ, is_active, created_at, updated_at)

tipe_kamar (id_tipe_kamar PK, nama_tipe UQ, kapasitas, harga_per_malam, deskripsi)

fasilitas (id_fasilitas PK, nama_fasilitas UQ, deskripsi)
kamar (id_kamar PK, id_tipe_kamar FK→tipe_kamar, nomor_kamar UQ, lantai, status_kamar)
```

```

reservasi (id_reservasi PK, id_tamu FK→tamu, id_pegawai FK→pegawai,
          tanggal_reservasi, tanggal_checkin_rencana,
          tanggal_checkout_rencana,
          status_reservasi)

detail_reservasi (id_detail_reservasi PK, id_reservasi FK→reservasi,
                  id_kamar FK→kamar, jumlah_malam, harga_per_malam,
                  subtotal GENERATED, UQ(id_reservasi, id_kamar))

pembayaran (id_pembayaran PK, id_reservasi FK→reservasi,
            tanggal_pembayaran,
            jumlah_bayar, metode_pembayaran, status_pembayaran)

checkin (id_checkin PK, id_reservasi FK→reservasi UQ,
         waktu_checkin, id_pegawai FK→pegawai, catatan)

checkout (id_checkout PK, id_reservasi FK→reservasi UQ,
          waktu_checkout, id_pegawai FK→pegawai, biaya_tambahan,
          catatan)

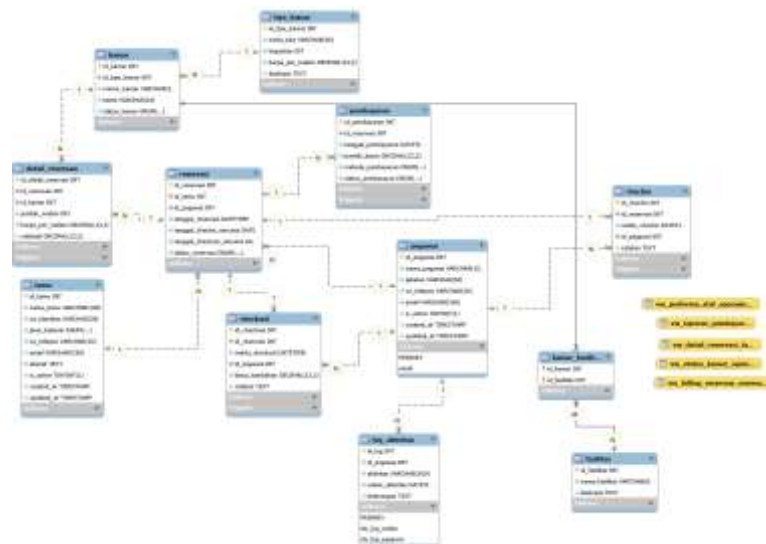
kamar_fasilitas (id_kamar FK→kamar, id_fasilitas FK→fasilitas,
                PK(id_kamar, id_fasilitas))

log_aktivitas (id_log PK, id_pegawai FK→pegawai NULL,
              aktivitas, waktu_aktivitas, keterangan)

```

3.3 Entity Relationship Diagram (ERD)

ERD dirancang menggunakan notasi Crow's Foot dan mencakup 12 entitas utama yang saling berelasi.



Gambar 3.1 Entity Relationship Diagram (ERD) Sistem Basis Data Reservasi Hotel

3.4 Relasi Utama Entitas dan Penjelasan Kardinalitas

1. tamu → reservasi : 1 : N (Satu tamu dapat melakukan banyak reservasi, sedangkan satu reservasi hanya dimiliki oleh satu tamu)
2. pegawai → reservasi : 1 : N (Satu pegawai dapat menangani banyak reservasi, sedangkan satu reservasi hanya ditangani oleh satu pegawai)
3. tipe_kamar → kamar : 1 : N (Satu tipe kamar dapat dimiliki oleh banyak kamar, sedangkan satu kamar hanya memiliki satu tipe kamar)
4. reservasi → detail_reservasi : 1 : N (Satu reservasi dapat memiliki beberapa detail reservasi, sedangkan satu detail reservasi hanya terkait dengan satu reservasi)
5. kamar → detail_reservasi : 1 : N (Satu kamar dapat muncul pada banyak detail reservasi dalam waktu yang berbeda, sedangkan satu detail reservasi hanya merujuk pada satu kamar)
6. reservasi → pembayaran : 1 : N (Satu reservasi dapat memiliki beberapa transaksi pembayaran, sedangkan satu pembayaran hanya terkait dengan satu reservasi)
7. reservasi → checkin : 1 : 1 (Satu reservasi hanya memiliki satu data check-in dan satu data check-in hanya terkait dengan satu reservasi)
8. reservasi → checkout : 1 : 1 (Satu reservasi hanya memiliki satu data check-out dan satu data check-out hanya terkait dengan satu reservasi)
9. pegawai → checkin : 1 : N (Satu pegawai dapat mencatat banyak proses check-in, sedangkan satu data check-in hanya dicatat oleh satu pegawai yang bertugas)
10. pegawai → checkout : 1 : N (Satu pegawai dapat mencatat banyak proses check-out, sedangkan satu data check-out hanya dicatat oleh satu pegawai yang bertugas)
11. pegawai → log aktivitas : 1 : N (Satu pegawai dapat menghasilkan banyak catatan aktivitas, sedangkan satu log aktivitas hanya dimiliki oleh satu pegawai)
12. kamar → kamar fasilitas : 1 : N melalui tabel kamar_fasilitas (Satu kamar dapat memiliki banyak data pada tabel KAMAR_FASILITAS, sedangkan satu data KAMAR_FASILITAS hanya terkait dengan satu kamar)

13. fasilitas → kamar fasilitas : 1 : N (Satu fasilitas dapat digunakan oleh banyak kamar, sedangkan satu data KAMAR_FASILITAS hanya terkait dengan satu fasilitas)

Relasi antara entitas KAMAR dan FASILITAS sebenarnya merupakan relasi Many-to-Many (M:N). Untuk mengimplementasikan relasi tersebut pada basis data relasional, digunakan entitas asosiasi KAMAR_FASILITAS yang memecah hubungan M:N menjadi dua relasi 1:N, yaitu KAMAR–KAMAR_FASILITAS dan FASILITAS–KAMAR_FASILITAS

3.5 Normalisasi

Normalisasi dilakukan untuk memastikan rancangan database Sistem Reservasi Hotel memiliki struktur yang efisien, konsisten, dan bebas dari duplikasi data yang tidak diperlukan. Proses ini membantu menghindari anomali insert, update, dan delete serta memudahkan penerapan relasi antar tabel.

Alur Normalisasi:

UNF -> 1NF -> 2NF -> 3NF

Tahap	Fokus Utama	Hasil
UNF	Mengidentifikasi data belum normal, repeating group, dan redundansi.	Data awal reservasi hotel masih bercampur.
1NF	Menghilangkan repeating group dan membuat nilai atomik.	Data dipisahkan agar setiap kolom berisi satu nilai.
2NF	Menghilangkan partial dependency.	Data master dan transaksi mulai dipisahkan ke tabel berbeda.
3NF	Menghilangkan transitive dependency.	Database final terdiri dari 12 tabel yang saling berelasi.

Functional Dependency Utama

```
id_tamu -> nama_tamu, no_identitas, email
id_tamu -> nama_tamu, no_identitas, email
id_pegawai -> nama_pegawai, jabatan
id_tipe_kamar -> nama_tipe, kapasitas, harga_per_malam
id_kamar -> id_tipe_kamar, nomor_kamar
id_reservasi -> id_tamu, id_pegawai, tanggal_reservasi
id_pembayaran -> jumlah_bayar, metode_pembayaran
```

Functional dependency tersebut menjadi dasar pemisahan tabel agar setiap atribut ditempatkan pada entitas yang tepat.

Daftar File Normalisasi:

File	Isi
UNF.md	Definisi UNF, contoh tabel sebelum normalisasi, repeating group, dan redundansi data.
1NF.md	Definisi 1NF, transformasi dari UNF ke 1NF, contoh tabel 1NF, dan nilai atomik.
2NF.md	Definisi 2NF, partial dependency, transformasi dari 1NF ke 2NF, dan contoh tabel hasil 2NF.
3NF.md	Definisi 3NF, transitive dependency, transformasi dari 2NF ke 3NF, dan hasil akhir database.

Ringkasan Hasil Akhir

Hasil akhir normalisasi menghasilkan 12 tabel:

1. tamu
2. pegawai
3. tipe_kamar
4. fasilitas
5. kamar
6. reservasi
7. detail_reservasi
8. pembayaran
9. checkin
10. checkout
11. kamar_fasilitas
12. log_aktivitas

Database telah memenuhi 3NF karena seluruh atribut non-key bergantung langsung pada primary key tabel masing-masing dan tidak terdapat ketergantungan transitif yang melanggar prinsip normalisasi.

1. UNF (Unnormalized Form)

Unnormalized Form (UNF) adalah bentuk data awal yang belum mengikuti aturan normalisasi. Pada tahap ini, data masih dapat mengandung atribut gabungan,

nilai multivalue, repeating group, dan redundansi data. Bentuk UNF sering muncul dari formulir transaksi atau catatan operasional yang masih mencampur beberapa informasi dalam satu baris.

Pada studi kasus Sistem Reservasi Hotel, data awal reservasi dapat berisi data tamu, kamar, tipe kamar, fasilitas, tanggal menginap, pembayaran, dan pegawai pencatat dalam satu tabel besar. Bentuk ini belum ideal untuk database relasional karena sulit menerapkan primary key, foreign key, dan constraint secara konsisten.

Tabel Sebelum Normalisasi

Contoh tabel UNF berikut menggambarkan data reservasi hotel sebelum proses normalisasi.

id reservasi	nama tamu	no identitas	pegawai	kamar	tipe kamar	fasilitas	tanggal checkin	tanggal checkout	pembayaran
R001	Andi Pratama	1001	Rani Kartika	101, 102	Standard, Standard	Wi-Fi, AC, TV LED	10/05/2026	12/05/2026	Rp700.000, Lunas
R002	Siti Rahmawati	1002	Doni Saputra	201	Superior	Wi-Fi, AC, TV LED, Breakfast	11/05/2026	14/05/2026	Rp1.350.000, Lunas
R003	Budi Santoso	1003	Fajar Nugroho	301	Deluxe	Mini Bar, Room Service	12/05/2026	14/05/2026	Rp1.200.000, Lunas

Repeating Group

Repeating group terjadi karena satu kolom menyimpan lebih dari satu nilai. Contohnya:

- Kolom `kamar` berisi 101, 102.
- Kolom `tipe_kamar` berisi lebih dari satu tipe untuk satu reservasi.
- Kolom `fasilitas` berisi beberapa fasilitas dalam satu sel.
- Kolom `pembayaran` menggabungkan metode pembayaran, nominal, dan status pembayaran.

Repeating group membuat data sulit dicari, dihitung, dan divalidasi menggunakan SQL.

Redundansi Data

Redundansi data terjadi ketika informasi yang sama disimpan berulang. Contohnya:

- Nama dan nomor identitas tamu dapat berulang pada banyak transaksi.
- Nama tipe kamar dan fasilitas dapat ditulis berulang pada setiap reservasi.
- Informasi pembayaran dapat bercampur dengan data reservasi sehingga sulit dipisahkan.

Redundansi meningkatkan risiko inkonsistensi. Misalnya, jika nama fasilitas berubah dari TV LED menjadi Smart TV, perubahan harus dilakukan pada banyak baris.

Permasalahan pada UNF

Permasalahan	Dampak
Data tidak atomik	Kolom berisi banyak nilai sehingga sulit difilter dan dihitung.
Repeating group	Satu reservasi dapat menyimpan banyak kamar dan fasilitas dalam satu kolom.
Redundansi data	Data tamu, kamar, tipe kamar, dan fasilitas berulang.
Risiko inkonsistensi	Perubahan data master dapat tidak seragam.
Sulit menerapkan key	Primary key dan foreign key sulit dirancang karena data masih bercampur.
Sulit membuat laporan	Query laporan menjadi kompleks dan rawan kesalahan.

Tabel Sesudah Normalisasi

Data UNF perlu diproses ke tahap 1NF dengan memecah nilai multivalue menjadi nilai atomik. Hasil tahap berikutnya akan memisahkan informasi menjadi tabel seperti `tamu`, `pegawai`, `kamar`, `fasilitas`, `reservasi`, dan `pembayaran`.

Kesimpulan

UNF merupakan bentuk awal data yang belum layak dijadikan struktur database final. Tahap ini penting untuk mengidentifikasi masalah utama seperti repeating group dan redundansi data sebelum dilakukan transformasi ke 1NF.

2. 1NF (First Normal Form)

First Normal Form (1NF) adalah tahap normalisasi yang memastikan setiap atribut berisi nilai atomik. Nilai atomik berarti satu kolom hanya boleh menyimpan satu nilai, bukan daftar nilai atau gabungan beberapa informasi.

Suatu tabel memenuhi 1NF apabila setiap kolom bernilai atomik, tidak terdapat repeating group, setiap baris dapat diidentifikasi dengan primary key atau candidate key dan data multivalue dipisahkan menjadi baris berbeda.

Transformasi dari UNF ke 1NF

Pada bentuk UNF, kolom seperti `kamar`, `fasilitas`, dan `pembayaran` masih berisi

nilai gabungan. Untuk mencapai 1NF, data tersebut dipisahkan sehingga setiap baris hanya berisi satu kamar dan satu fasilitas.

Tabel Sebelum Normalisasi

id_reservasi	nama_tamu	kamar	fasilitas	pembayaran
R001	Andi Pratama	101, 102	Wi-Fi, AC, TV LED	Transfer, Rp700.000, Lunas

Tabel tersebut belum memenuhi 1NF karena kolom kamar, fasilitas, dan pembayaran tidak atomik.

Contoh Tabel Hasil 1NF:

id_reservasi	nama_tamu	no_identitas	nomor_kamar	tipe_kamar	fasilitas	tanggal_checkin	tanggal_checkout	metode_pembayaran	jumlah_bayar	status_pembayaran
R001	Andi Pratama	1001	101	Standard	Wi-Fi	10/05/2026	12/05/2026	Transfer	700000	Lunas
R001	Andi Pratama	1001	101	Standard	AC	10/05/2026	12/05/2026	Transfer	700000	Lunas
R001	Andi Pratama	1001	101	Standard	TV LED	10/05/2026	12/05/2026	Transfer	700000	Lunas
R001	Andi Pratama	1001	102	Standard	Wi-Fi	10/05/2026	12/05/2026	Transfer	700000	Lunas

Penjelasan Nilai Atomik

Nilai atomik diterapkan dengan cara:

- Kolom `nomor_kamar` hanya menyimpan satu nomor kamar.
- Kolom `fasilitas` hanya menyimpan satu nama fasilitas.
- Kolom `pembayaran` dipisah menjadi `metode_pembayaran`, `jumlah_bayar`, dan `status_pembayaran`.

Setiap baris merepresentasikan satu fakta yang lebih spesifik.

Functional Dependency pada 1NF

Pada tahap 1NF, beberapa functional dependency mulai terlihat:

`id_tamu -> nama_tamu, no_identitas, email`

`id_pegawai -> nama_pegawai, jabatan`

`id_kamar -> nomor_kamar, tipe_kamar`

`id_reservasi -> id_tamu, tanggal_checkin, tanggal_checkout`

`id_pembayaran -> jumlah_bayar, metode_pembayaran, status_pembayaran`

Namun, data tersebut masih bercampur dalam satu tabel besar sehingga masih terjadi partial dependency dan redundansi.

Tabel Sesudah Normalisasi

Setelah 1NF, data belum langsung menjadi tabel final. Tahap ini menghasilkan data atomik yang kemudian dipisahkan lebih lanjut pada 2NF menjadi tabel seperti:

- tamu
- reservasi
- kamar
- tipe_kamar
- fasilitas
- pembayaran

Kesimpulan

1NF berhasil menghilangkan repeating group dan memastikan nilai pada setiap kolom bersifat atomik. Namun, tabel 1NF masih memiliki duplikasi data dan partial dependency sehingga perlu dilanjutkan ke 2NF.

3. 2NF (Second Normal Form)

Second Normal Form (2NF) adalah tahap normalisasi yang memastikan tabel sudah memenuhi 1NF dan seluruh atribut non-key bergantung penuh pada primary key. 2NF berfokus pada penghapusan partial dependency.

Partial dependency terjadi ketika atribut non-key hanya bergantung pada sebagian dari composite key, bukan pada keseluruhan key.

Partial Dependency

Pada bentuk 1NF, satu baris dapat diidentifikasi oleh kombinasi data seperti `id_reservasi`, `id_kamar`, dan `id_fasilitas`. Masalah muncul karena beberapa atribut hanya bergantung pada sebagian dari kombinasi tersebut.

Contoh partial dependency:

```
id_tamu -> nama_tamu, no_identitas, email
id_pegawai -> nama_pegawai, jabatan
id_tipe_kamar -> nama_tipe, kapasitas, harga_per_malam
id_kamar -> id_tipe_kamar, nomor_kamar
id_fasilitas -> nama_fasilitas, deskripsi
id_reservasi -> id_tamu, id_pegawai, tanggal_reservasi
id_pembayaran -> jumlah_bayar, metode_pembayaran
```

Ketergantungan tersebut harus dipisahkan agar setiap tabel hanya menyimpan atribut yang benar-benar bergantung pada primary key tabelnya.

Transformasi dari 1NF ke 2NF

Transformasi dilakukan dengan memecah tabel besar hasil 1NF menjadi beberapa tabel

berdasarkan kelompok ketergantungan data.

Tabel Sebelum Normalisasi

id_reservasi	id_tamu	nama_tamu	id_kamar	nomor_kamar	id_tipe_kamar	nama_tipe	id_fasilitas	nama_fasilitas	jumlah_bayar
R001	T001	Andi Pratama	K001	101	TK001	Standard	F001	Wi-Fi	700000
R001	T001	Andi Pratama	K001	101	TK001	Standard	F002	AC	700000

Masalah utama tabel tersebut adalah data tamu, kamar, tipe kamar, fasilitas, dan pembayaran berulang pada banyak baris.

Contoh Tabel Hasil 2NF

Tabel tamu

id_tamu	nama_tamu	no_identitas	email
T001	Andi Pratama	1001	<u>andi.pratama@example.com</u>

Tabel tipe_kamar

id_tipe_kamar	nama_tipe	kapasitas	harga_per_malam
TK001	Standard	2	350000

Tabel kamar

id_kamar	id_tipe_kamar	nomor_kamar	status_kamar
K001	TK001	101	Tersedia

Tabel fasilitas

id_fasilitas	nama_fasilitas	deskripsi
F001	Wi-Fi	Akses internet nirkabel
F002	AC	Pendingin ruangan

Tabel reservasi

id_reservasi	id_tamu	id_pegawai	tanggal_reservasi	tanggal_checkin_rencana	tanggal_checkout_rencana
R001	T001	P001	01/05/2026	10/05/2026	12/05/2026

Tabel detail_reservasi

id_detail_reservasi	id_reservasi	id_kamar	jumlah_malam	subtotal
DR001	R001	K001	2	700000

Tabel kamar_fasilitas

id_kamar	id_fasilitas
K001	F001
K001	F002

Functional Dependency Setelah 2NF

id_tamu -> nama_tamu, no_identitas, jenis_kelamin, no_telepon, email, alamat
 id_pegawai -> nama_pegawai, jabatan, no_telepon, email
 id_tipe_kamar -> nama_tipe, kapasitas, harga_per_malam, deskripsi
 id_kamar -> id_tipe_kamar, nomor_kamar, lantai, status_kamar
 id_fasilitas -> nama_fasilitas, deskripsi
 id_reservasi -> id_tamu, id_pegawai, tanggal_reservasi, tanggal_checkin_rencana, tanggal_checkout_rencana, status_reservasi
 id_detail_reservasi -> id_reservasi, id_kamar, jumlah_malam, harga_per_malam, subtotal

Tabel Sesudah Normalisasi

Pada tahap 2NF, data dipisahkan menjadi tabel:

- tamu
- pegawai
- tipe_kamar
- kamar
- fasilitas
- reservasi
- detail_reservasi

- kamar_fasilitas

Kesimpulan

2NF menghilangkan partial dependency dengan memisahkan atribut ke tabel yang sesuai. Hasilnya, data lebih konsisten, duplikasi berkurang, dan struktur database lebih siap untuk disempurnakan ke 3NF.

4. 3NF (Third Normal Form)

Third Normal Form (3NF) adalah tahap normalisasi yang memastikan tabel sudah memenuhi 2NF dan tidak memiliki transitive dependency. Transitive dependency terjadi ketika atribut non-key bergantung pada atribut non-key lain, bukan langsung pada primary key.

Transitive Dependency

Contoh transitive dependency pada data reservasi hotel:

```
id_reservasi -> id_tamu -> nama_tamu, no_identitas, email
id_reservasi -> id_pegawai -> nama_pegawai, jabatan
id_kamar -> id_tipe_kamar -> nama_tipe, kapasitas, harga_per_malam
id_reservasi -> pembayaran -> jumlah_bayar, metode_pembayaran
```

Ketergantungan tersebut dihilangkan dengan menempatkan data tamu, pegawai, tipe kamar, pembayaran, check-in, dan check-out pada tabel masing-masing.

Transformasi dari 2NF ke 3NF

Transformasi dari 2NF ke 3NF dilakukan dengan:

1. Memastikan data tamu hanya disimpan pada tabel tamu.
2. Memastikan data pegawai hanya disimpan pada tabel pegawai.
3. Memastikan informasi tipe kamar hanya disimpan pada tabel tipe_kamar.
4. Memisahkan data pembayaran ke tabel pembayaran.
5. Memisahkan data check-in ke tabel checkin.
6. Memisahkan data check-out ke tabel checkout.
7. Memisahkan aktivitas sistem ke tabel log_aktivitas.

Tabel Sebelum Normalisasi

id_reservasi	id_tamu	nama_tamu	id_kamar	id_tipe_kamar	nama_tipe	harga_per_malam	jumlah_bayar	metode_pembayaran
R001	T001	Andi Pratama	K001	TK001	Standard	350000	700000	Transfer

Tabel tersebut masih mengandung transitive dependency karena nama tamu bergantung pada id_tamu, nama tipe kamar bergantung pada id_tipe_kamar, dan pembayaran

memiliki atribut yang seharusnya disimpan pada tabel pembayaran.

Hasil Akhir Database

No	Nama Tabel	Fungsi
1	tamu	Menyimpan data identitas tamu.
2	pegawai	Menyimpan data pegawai hotel.
3	tipe_kamar	Menyimpan kategori kamar, kapasitas, dan harga.
4	fasilitas	Menyimpan daftar fasilitas hotel.
5	kamar	Menyimpan data kamar fisik hotel.
6	reservasi	Menyimpan transaksi reservasi.
7	detail_reservasi	Menyimpan rincian kamar pada reservasi.
8	pembayaran	Menyimpan data pembayaran reservasi.
9	checkin	Menyimpan data realisasi check-in.
10	checkout	Menyimpan data realisasi check-out.
11	kamar_fasilitas	Menyimpan relasi many-to-many antara kamar dan fasilitas.
12	log_aktivitas	Menyimpan catatan aktivitas sistem.

Functional Dependency Final

id_tamu -> nama_tamu, no_identitas, jenis_kelamin, no_telepon, email, alamat, created_at
id_pegawai -> nama_pegawai, jabatan, no_telepon, email, created_at
id_tipe_kamar -> nama_tipe, kapasitas, harga_per_malam, deskripsi
id_fasilitas -> nama_fasilitas, deskripsi
id_kamar -> id_tipe_kamar, nomor_kamar, lantai, status_kamar
id_reservasi -> id_tamu, id_pegawai, tanggal_reservasi, tanggal_checkin_rencana, tanggal_checkout_rencana, status_reservasi
id_detail_reservasi -> id_reservasi, id_kamar, jumlah_malam, harga_per_malam, subtotal
id_pembayaran -> id_reservasi, tanggal_pembayaran, jumlah_bayar, metode_pembayaran, status_pembayaran
id_checkin -> id_reservasi, waktu_checkin, id_pegawai, catatan
id_checkout -> id_reservasi, waktu_checkout, id_pegawai, biaya_tambahan, catatan
(id_kamar, id_fasilitas) -> relasi_fasilitas_pada_kamar
id_log -> id_pegawai, aktivitas, waktu_aktivitas, keterangan

Tabel Sesudah Normalisasi

tamu(id_tamu PK, nama_tamu, no_identitas UK, jenis_kelamin,

```

no_telepon, email UK, alamat, created_at)
    pegawai(id_pegawai PK, nama_pegawai, jabatan, no_telepon, email
UK, created_at)
    tipe_kamar(id_tipe_kamar PK, nama_tipe UK, kapasitas,
harga_per_malam, deskripsi)
    fasilitas(id_fasilitas PK, nama_fasilitas UK, deskripsi)
    kamar(id_kamar PK, id_tipe_kamar FK, nomor_kamar UK, lantai,
status_kamar)
    reservasi(id_reservasi PK, id_tamu FK, id_pegawai FK,
tanggal_reservasi, tanggal_checkin_rencana, tanggal_checkout_rencana,
status_reservasi)
    detail_reservasi(id_detail_reservasi PK, id_reservasi FK,
id_kamar FK, jumlah_malam, harga_per_malam, subtotal)
    pembayaran(id_pembayaran PK, id_reservasi FK, tanggal_pembayaran,
jumlah_bayar, metode_pembayaran, status_pembayaran)
    checkin(id_checkin PK, id_reservasi FK/UK, waktu_checkin,
id_pegawai FK, catatan)
    checkout(id_checkout PK, id_reservasi FK/UK, waktu_checkout,
id_pegawai FK, biaya_tambahan, catatan)
    kamar_fasilitas(id_kamar PK/FK, id_fasilitas PK/FK)
    log_aktivitas(id_log PK, id_pegawai FK, aktivitas,
waktu_aktivitas, keterangan)

```

Manfaat 3NF

- a. Mengurangi duplikasi data.
- b. Menjaga konsistensi data master dan transaksi.
- c. Mempermudah proses update, insert, dan delete.
- d. Mempermudah penerapan primary key dan foreign key.
- e. Memudahkan pembuatan laporan reservasi, pembayaran, dan aktivitas sistem.
- f. Meningkatkan kualitas rancangan database untuk kebutuhan akademik dan implementasi.

Kesimpulan

Database Sistem Reservasi Hotel telah mencapai 3NF karena setiap atribut non-key bergantung langsung pada primary key tabelnya dan tidak terdapat transitive dependency. Struktur final database sudah sesuai dengan kebutuhan project Basis Data.

3.6 Data Dictionary

1. Tabel tamu

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_tamu	INT	–	PK, AUTO_INCREMENT	Identifier unik tamu
nama_tamu	VARCHAR	100	NOT NULL	Nama

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
				lengkap tamu
no_identitas	VARCHAR	30	NOT NULL, UNIQUE	Nomor KTP/Paspor tamu
jenis_kelamin	ENUM	–	NOT NULL	Laki-laki / Perempuan
no_telepon	VARCHAR	20	NOT NULL	Nomor telepon aktif
email	VARCHAR	100	UNIQUE	Alamat email tamu
alamat	TEXT	–	NOT NULL	Alamat lengkap domisili
is_active	TINYINT(1)	–	DEFAULT 1	Flag soft delete: 1=Aktif, 0=Nonaktif
created_at	TIMESTAMP	–	DEFAULT CURRENT_TIMESTAMP	Waktu data dibuat
updated_at	TIMESTAMP	–	ON UPDATE CURRENT_TIMESTAMP	Waktu data diperbarui

2. Tabel pegawai

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_pegawai	INT	–	PK, AUTO_INCREMENT	Identifer unik pegawai
nama_pegawai	VARCHAR	100	NOT NULL	Nama lengkap pegawai
jabatan	VARCHAR	50	NOT NULL	Posisi/jabatan di hotel
no_telepon	VARCHAR	20	NOT NULL	Nomor telepon pegawai
email	VARCHAR	100	NOT NULL, UNIQUE	Email internal pegawai
is_active	TINYINT(1)	–	DEFAULT 1	Status aktif pegawai
created_at	TIMESTAMP	–	DEFAULT CURRENT_TIMESTAMP	Waktu data dibuat

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
updated_at	TIMESTAMP	–	ON UPDATE CURRENT_TIMESTAMP	Waktu data diperbarui

3. Tabel tipe kamar

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_tipe_kamar	INT	–	PK, AUTO_INCREMENT	Identifer unik tipe kamar
nama_tipe	VARCHAR	50	NOT NULL, UNIQUE	Nama tipe (Standard/Superior/Deluxe/Family/Suite)
kapasitas	INT	–	NOT NULL, CHECK > 0	Jumlah maksimal tamu
harga_per_malam	DECIMAL(12,2)	–	NOT NULL, CHECK >= 0	Harga per malam dalam rupiah
deskripsi	TEXT	–	–	Deskripsi singkat tipe kamar

4. Tabel fasilitas

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_fasilitas	INT	–	PK, AUTO_INCREMENT	Identifer unik fasilitas
nama_fasilitas	VARCHAR	80	NOT NULL, UNIQUE	Nama fasilitas (Wi-Fi, AC, dll.)
deskripsi	TEXT	–	–	Penjelasan fasilitas

5. Tabel kamar

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_kamar	INT	–	PK, AUTO_INCREMENT	Identifer unik kamar
id_tipe_kamar	INT	–	FK → tipe_kamar, NOT NULL	Referensi tipe kamar
nomor_kamar	VARCHAR	10	NOT NULL, UNIQUE	Nomor kamar (misal: 101, 501)
lantai	VARCHAR	10	NOT NULL	Lantai tempat kamar berada
status_kamar	ENUM	–	NOT NULL,	Tersedia/Dipesan/Teri

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
r			DEFAULT 'Tersedia'	si/Perawatan

6. Tabel reservasi

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_reservasi	INT	—	PK, AUTO_INCREMENT	Identifer unik reservasi
id_tamu	INT	—	FK → tamu, NOT NULL	Tamu yang melakukan reservasi
id_pegawai	INT	—	FK → pegawai, NOT NULL	Pegawai yang memproses
tanggal_reservasi	DATETIME	—	NOT NULL, DEFAULT NOW()	Waktu reservasi dibuat
tanggal_checkin_rencana	DATE	—	NOT NULL	Tanggal check-in yang direncanakan
tanggal_checkout_rencana	DATE	—	NOT NULL, CHECK > checkin	Tanggal check-out yang direncanakan
status_reservasi	ENUM	—	NOT NULL, DEFAULT 'Menunggu'	Menunggu/Dikonfirmasi/ Check-in/Selesai/Dibatalkan

7. Tabel detail reservasi

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_detail_reservasi	INT	—	PK, AUTO_INCREMENT	Identifer unik detail
id_reservasi	INT	—	FK → reservasi, NOT NULL	Reservasi induk
id_kamar	INT	—	FK → kamar, NOT NULL	Kamar yang dipesan
jumlah_malam	INT	—	NOT NULL, CHECK > 0	Durasi menginap (malam)
harga_per_malam	DECIMAL(12,2)	—	NOT NULL, CHECK >= 0	Harga snapshot saat pemesanan

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
subtotal	DECIMAL(12,2)	–	GENERATED ALWAYS AS (jml × harga)	Subtotal otomatis: jumlah_malam × harga

8. Tabel pembayaran

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_pembayaran	INT	–	PK, AUTO_INCREMENT	Identifer unik pembayaran
id_reservasi	INT	–	FK → reservasi, NOT NULL	Reservasi yang dibayar
tanggal_pembayaran	DATETIME	–	NOT NULL, DEFAULT NOW()	Waktu pembayaran dilakukan
jumlah_bayar	DECIMAL(12,2)	–	NOT NULL, CHECK > 0	Nominal yang dibayarkan
metode_pembayaran	ENUM	–	NOT NULL	Tunai/Transfer/Kartu Kredit/E-Wallet
status_pembayaran	ENUM	–	NOT NULL, DEFAULT 'Pending'	Pending/Lunas/Gagal/Refund

9. Tabel checkin

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_checkin	INT	–	PK, AUTO_INCREMENT	Identifer unik check-in
id_reservasi	INT	–	FK → reservasi, UNIQUE	Reservasi (satu reservasi = satu check-in)
waktu_checkin	DATETIME	–	NOT NULL, DEFAULT NOW()	Waktu aktual check-in
id_pegawai	INT	–	FK → pegawai, NOT NULL	Pegawai yang memproses check-in
catatan	TEXT	–	–	Catatan khusus saat check-in

10. Tabel checkout

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_checkout	INT	–	PK, AUTO_INCREMENT	Identifer unik check-out
id_reservasi	INT	–	FK → reservasi, UNIQUE	Reservasi (satu reservasi = satu check-out)
waktu_checkout	DATETIME	–	NOT NULL, DEFAULT NOW()	Waktu aktual check-out
id_pegawai	INT	–	FK → pegawai, NOT NULL	Pegawai yang memproses check-out
biaya_tambahan	DECIMAL(12,2)	–	NOT NULL, DEFAULT 0, CHECK ≥ 0	Biaya ekstra (laundry, minibar, dll.)
catatan	TEXT	–	–	Catatan khusus saat check-out

11. Tabel kamar fasilitas

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_kamar	INT	–	PK (composite), FK → kamar	Kamar
id_fasilitas	INT	–	PK (composite), FK → fasilitas	Fasilitas yang dimiliki kamar

12. Tabel log aktivitas

Kolom	Tipe Data	Panjang	Constraint	Deskripsi
id_log	INT	–	PK, AUTO_INCREMENT	Identifer unik log
id_pegawai	INT	–	FK → pegawai, NULL	Pegawai pelaku (NULL jika dihapus)
aktivitas	VARCHAR	100	NOT NULL	Jenis aktivitas yang dilakukan
waktu_aktivitas	DATETIME	–	NOT NULL, DEFAULT NOW()	Waktu aktivitas terjadi
keterangan	TEXT	–	–	Detail keterangan aktivitas

BAB 4

IMPLEMENTASI

4.1 Lingkungan Implementasi

Sistem basis data diimplementasikan menggunakan MySQL Server 8.0 sebagai Database Management System (DBMS) untuk mendukung transaksi, foreign key, dan integritas referensial. Database dibuat dengan nama `hotel_reservation_db` menggunakan karakter set `utf8mb4` dan collation `utf8mb4_unicode_ci`. MySQL dipilih karena mendukung implementasi basis data relasional, memiliki performa yang baik, serta menyediakan fitur lengkap seperti foreign key, trigger, procedure, dan function.

4.2 Implementasi DDL (Data Definition Language)

DDL digunakan untuk membentuk struktur basis data berupa tabel, primary key, foreign key, constraint, dan index. Script DDL mencakup pembuatan database, 12 tabel, constraint, index, dan 2 trigger otomatisasi. Berikut penjelasan setiap komponen implementasi:

1. Inisialisasi Database

```
CREATE DATABASE IF NOT EXISTS hotel_reservation_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE hotel_reservation_db;
```

Database dibuat dengan character set `utf8mb4` untuk mendukung penyimpanan karakter Unicode (termasuk emoji dan karakter khusus). Klausa `IF NOT EXISTS` mencegah error apabila database sudah ada.

2. Tabel Master: tamu, pegawai, tipe_kamar, fasilitas

Keempat tabel ini berfungsi sebagai data referensi (master data) yang menjadi acuan tabel-tabel transaksi.

```
-- 1. TABEL MASTER TAMU
-- =====
CREATE TABLE tamu (
    id_tamu INT AUTO_INCREMENT PRIMARY KEY,
    nama_tamu VARCHAR(100) NOT NULL,
    no_identitas VARCHAR(30) NOT NULL UNIQUE,
    jenis_kelamin ENUM('Laki-laki', 'Perempuan') NOT NULL,
    no_telepon VARCHAR(20) NOT NULL,
```

```

        email VARCHAR(100) UNIQUE,
        alamat TEXT NOT NULL,
        is_active TINYINT(1) DEFAULT 1 COMMENT '1 = Aktif, 0 = Soft Delete',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
    ) ENGINE=InnoDB;

-- =====
-- 2. TABEL MASTER PEGAWAI
-- =====
CREATE TABLE pegawai (
    id_pegawai INT AUTO_INCREMENT PRIMARY KEY,
    nama_pegawai VARCHAR(100) NOT NULL,
    jabatan VARCHAR(50) NOT NULL,
    no_telepon VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    is_active TINYINT(1) DEFAULT 1 COMMENT '1 = Aktif, 0 = Non-Aktif',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- =====
-- 3. TABEL TIPE KAMAR
-- =====
CREATE TABLE tipe_kamar (
    id_tipe_kamar INT AUTO_INCREMENT PRIMARY KEY,
    nama_tipe VARCHAR(50) NOT NULL UNIQUE,
    kapasitas INT NOT NULL CHECK (kapasitas > 0),
    harga_per_malam DECIMAL(12,2) NOT NULL CHECK (harga_per_malam >= 0),
    deskripsi TEXT
) ENGINE=InnoDB;

-- =====
-- 4. TABEL FASILITAS
-- =====
CREATE TABLE fasilitas (
    id_fasilitas INT AUTO_INCREMENT PRIMARY KEY,
    nama_fasilitas VARCHAR(80) NOT NULL UNIQUE,
    deskripsi TEXT
) ENGINE=InnoDB;

```

Poin desain penting: `is_active` memungkinkan soft delete, tamu dinonaktifkan tanpa dihapus secara fisik, sehingga histori reservasi tetap terjaga. Kolom audit (`created_at`, `updated_at`) diperbarui otomatis oleh MySQL.

No	Nama Tabel	Fungsi
1	tamu	Menyimpan data identitas tamu.
2	pegawai	Menyimpan data pegawai hotel.
3	tipe_kamar	Menyimpan kategori kamar, kapasitas, dan harga.
4	fasilitas	Menyimpan daftar fasilitas hotel.

3. Tabel Operasional: kamar, reservasi, detail_reservasi, pembayaran, checkin,checkout, kamar_fasilitas, log_aktivitas

a. Tabel kamar dengan index

```
-- 5. TABEL KAMAR
-- =====
CREATE TABLE kamar (
  id_kamar INT AUTO_INCREMENT PRIMARY KEY,
  id_tipe_kamar INT NOT NULL,
  nomor_kamar VARCHAR(10) NOT NULL UNIQUE,
  lantai VARCHAR(10) NOT NULL,
  status_kamar ENUM('Tersedia', 'Dipesan', 'Terisi', 'Perawatan')
NOT NULL DEFAULT 'Tersedia',
  CONSTRAINT fk_kamar_tipe FOREIGN KEY (id_tipe_kamar) REFERENCES
tipe_kamar(id_tipe_kamar)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  INDEX idx_kamar_tipe (id_tipe_kamar),
  INDEX idx_kamar_status (status_kamar)
) ENGINE=InnoDB;
```

Index dibuat pada kolom `id_tipe_kamar` dan `status_kamar` karena keduanya sering digunakan sebagai filter dalam query operasional (misal: mencari kamar yang tersedia berdasarkan tipe).

b. Tabel reservasi dengan CHECK Constraint

```
-- 6. TABEL RESERVASI (HEADER)
-- =====
CREATE TABLE reservasi (
  id_reservasi INT AUTO_INCREMENT PRIMARY KEY,
  id_tamu INT NOT NULL,
  id_pegawai INT NOT NULL,
  tanggal_reservasi DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  tanggal_checkin_rencana DATE NOT NULL,
  tanggal_checkout_rencana DATE NOT NULL,
  status_reservasi ENUM('Menunggu', 'Dikonfirmasi', 'Check-in',
'Selesai', 'Dibatalkan') NOT NULL DEFAULT 'Menunggu',
  CONSTRAINT fk_reservasi_tamu FOREIGN KEY (id_tamu) REFERENCES
tamu(id_tamu)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_reservasi_pegawai FOREIGN KEY (id_pegawai)
REFERENCES pegawai(id_pegawai)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_tanggal_reservasi CHECK (tanggal_checkout_rencana
> tanggal_checkin_rencana),
  INDEX idx_reservasi_tamu (id_tamu),
  INDEX idx_reservasi_pegawai (id_pegawai),
  INDEX idx_reservasi_status (status_reservasi)
) ENGINE=InnoDB;
```

CHECK constraint memastikan tanggal checkout selalu lebih besar dari tanggal checkin di tingkat database, sehingga validasi tidak hanya bergantung pada logika aplikasi.

c. Tabel detail_reservasi dengan Generated Column

```
-- 7. TABEL DETAIL RESERVASI
-- =====
CREATE TABLE detail_reservasi (
    id_detail_reservasi INT AUTO_INCREMENT PRIMARY KEY,
    id_reservasi INT NOT NULL,
    id_kamar INT NOT NULL,
    jumlah_malam INT NOT NULL CHECK (jumlah_malam > 0),
    harga_per_malam DECIMAL(12,2) NOT NULL CHECK (harga_per_malam >=
0),
    -- UPGRADE: Otomatis dihitung oleh database untuk akurasi data
    finansial
    subtotal DECIMAL(12,2) GENERATED ALWAYS AS (jumlah_malam *
harga_per_malam) STORED,
    CONSTRAINT fk_detail_reservasi FOREIGN KEY (id_reservasi)
REFERENCES reservasi(id_reservasi)
    ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_detail_kamar FOREIGN KEY (id_kamar) REFERENCES
kamar(id_kamar)
    ON UPDATE CASCADE ON DELETE RESTRICT,
    UNIQUE KEY uk_reservasi_kamar (id_reservasi, id_kamar),
    INDEX idx_detail_kamar (id_kamar)
) ENGINE=InnoDB;
```

Generated column STORED menyimpan nilai subtotal secara fisik di disk dan dihitung ulang otomatis setiap kali jumlah_malam atau harga_per_malam diubah. Ini mengeliminasi risiko ketidakkonsistenan bila perhitungan dilakukan di sisi aplikasi. UNIQUE KEY (id_reservasi, id_kamar) mencegah kamar yang sama dipesan dua kali dalam satu reservasi.

4. Tabel pembayaran

```
CREATE TABLE pembayaran (
    id_pembayaran INT AUTO_INCREMENT PRIMARY KEY,
    id_reservasi INT NOT NULL,
    tanggal_pembayaran DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    jumlah_bayar DECIMAL(12,2) NOT NULL CHECK (jumlah_bayar > 0),
    metode_pembayaran ENUM('Tunai', 'Transfer', 'Kartu Kredit', 'E-Wallet')
NOT NULL,
    status_pembayaran ENUM('Pending', 'Lunas', 'Gagal', 'Refund')
    NOT NULL DEFAULT 'Pending',
    CONSTRAINT fk_pembayaran_reservasi FOREIGN KEY (id_reservasi)
REFERENCES reservasi(id_reservasi) ON UPDATE CASCADE ON DELETE
CASCADE,
    INDEX idx_pembayaran_reservasi (id_reservasi),
    INDEX idx_pembayaran_status (status_pembayaran)
) ENGINE=InnoDB;
```

Tabel pembayaran didesain one-to-many terhadap reservasi — satu reservasi dapat memiliki lebih dari satu baris pembayaran untuk mengakomodasi skenario cicilan (partial payment). CHECK (jumlah_bayar > 0) memastikan tidak ada transaksi bernilai nol atau negatif yang masuk ke sistem. Status Refund pada ENUM digunakan saat reservasi dibatalkan dan uang tamu perlu dikembalikan, dipicu oleh stored procedure sp_batal_reservasi_otomatis. ON DELETE CASCADE memastikan baris pembayaran ikut terhapus apabila reservasi induknya dihapus, menjaga konsistensi data.

5. Tabel checkin

```
CREATE TABLE checkin (  
    id_checkin INT AUTO_INCREMENT PRIMARY KEY,  
    id_reservasi INT NOT NULL UNIQUE,  
    waktu_checkin DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    id_pegawai INT NOT NULL,  
    catatan TEXT,  
    CONSTRAINT fk_checkin_reservasi FOREIGN KEY (id_reservasi)  
        REFERENCES reservasi(id_reservasi) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    CONSTRAINT fk_checkin_pegawai FOREIGN KEY (id_pegawai)  
        REFERENCES pegawai(id_pegawai) ON UPDATE CASCADE ON DELETE  
RESTRICT  
) ENGINE=InnoDB;
```

Kolom `id_reservasi` diberi constraint `NOT NULL` sekaligus `UNIQUE`, yang secara efektif membentuk relasi one-to-one antara checkin dan reservasi — satu reservasi hanya dapat memiliki tepat satu catatan check-in aktual. Hal ini masuk akal secara logis bisnis: tamu tidak mungkin check-in dua kali untuk reservasi yang sama. `waktu_checkin` menggunakan `DATETIME` (bukan `DATE`) untuk mencatat jam dan menit kedatangan secara presisi, yang berguna untuk perhitungan biaya late check-in maupun pelaporan operasional. Foreign key ke pegawai menggunakan `ON DELETE RESTRICT` karena catatan siapa yang memproses check-in tidak boleh hilang hanya karena data pegawai tersebut dihapus.

6. Tabel checkout

```
CREATE TABLE checkout (  
    id_checkout INT AUTO_INCREMENT PRIMARY KEY,  
    id_reservasi INT NOT NULL UNIQUE,  
    waktu_checkout DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    id_pegawai INT NOT NULL,  
    biaya_tambahan DECIMAL(12,2) NOT NULL DEFAULT 0.00  
        CHECK (biaya_tambahan >= 0),  
    catatan TEXT,  
    CONSTRAINT fk_checkout_reservasi FOREIGN KEY (id_reservasi)  
        REFERENCES reservasi(id_reservasi) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    CONSTRAINT fk_checkout_pegawai FOREIGN KEY (id_pegawai)  
        REFERENCES pegawai(id_pegawai) ON UPDATE CASCADE ON DELETE  
RESTRICT  
) ENGINE=InnoDB;
```

Pola desain tabel checkout identik dengan checkin (relasi one-to-one via `UNIQUE` pada `id_reservasi`), dengan satu kolom tambahan yang krusial: `biaya_tambahan`. Kolom ini menampung biaya yang baru dapat diketahui nilainya di akhir masa inap, seperti minibar, laundry, atau biaya kerusakan fasilitas. Nilai defaultnya 0.00 sehingga jika tidak ada biaya tambahan, insert checkout tetap valid tanpa perlu mengisi kolom ini. `CHECK (biaya_tambahan >= 0)` memastikan nilainya tidak negatif. Kolom `biaya_tambahan` inilah

yang kemudian dijumlahkan dengan subtotal kamar oleh function `fn_total_pendapatan_reservasi` dan view `vw_billing_reservasi_summary` untuk menghasilkan grand total tagihan yang sesungguhnya.

7. Tabel kamar_fasilitas

```
CREATE TABLE kamar_fasilitas (  
    id_kamar INT NOT NULL,  
    id_fasilitas INT NOT NULL,  
    PRIMARY KEY (id_kamar, id_fasilitas),  
    CONSTRAINT fk_kf_kamar FOREIGN KEY (id_kamar)  
        REFERENCES kamar(id_kamar) ON UPDATE CASCADE ON DELETE CASCADE,  
    CONSTRAINT fk_kf_fasilitas FOREIGN KEY (id_fasilitas)  
        REFERENCES fasilitas(id_fasilitas) ON UPDATE CASCADE ON DELETE  
    CASCADE  
) ENGINE=InnoDB;
```

Tabel ini adalah tabel junction yang merepresentasikan relasi many-to-many antara kamar dan fasilitas. Relasi many-to-many tidak dapat dimodelkan langsung dengan foreign key biasa, sehingga diperlukan tabel perantara ini. Satu kamar dapat memiliki banyak fasilitas (Wi-Fi, AC, TV, dll.), dan satu fasilitas seperti Wi-Fi dapat dimiliki oleh banyak kamar sekaligus.

Primary key yang digunakan adalah primary key komposit, yakni gabungan (`id_kamar`, `id_fasilitas`). Ini memastikan kombinasi yang sama tidak dapat muncul dua kali, misalnya kamar 101 tidak bisa terdaftar memiliki Wi-Fi sebanyak dua kali. Kedua foreign key menggunakan ON DELETE CASCADE agar apabila sebuah kamar atau fasilitas dihapus, seluruh relasi yang melibatkannya ikut terhapus secara otomatis.

8. Tabel log_aktivitas

```
CREATE TABLE log_aktivitas (  
    id_log INT AUTO_INCREMENT PRIMARY KEY,  
    id_pegawai INT NULL,  
    aktivitas VARCHAR(100) NOT NULL,  
    waktu_aktivitas DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    keterangan TEXT,  
    CONSTRAINT fk_log_pegawai FOREIGN KEY (id_pegawai)  
        REFERENCES pegawai(id_pegawai) ON UPDATE CASCADE ON DELETE SET  
    NULL,  
    INDEX idx_log_waktu (waktu_aktivitas),  
    INDEX idx_log_pegawai (id_pegawai)  
) ENGINE=InnoDB;
```

Tabel ini mencatat jejak aktivitas seluruh pegawai di sistem sebagai mekanisme audit trail, siapa melakukan apa dan kapan. Tabel ini memiliki keputusan desain yang berbeda dari tabel lain: kolom `id_pegawai` dideklarasikan NULL (boleh kosong) dengan ON DELETE SET NULL pada foreign key-nya.

Alasan pemilihan ON DELETE SET NULL (bukan RESTRICT seperti tabel lain):

apabila seorang pegawai dihapus dari sistem karena resign atau alasan lain, histori log aktivitasnya tidak boleh ikut terhapus, itu akan merusak jejak audit dan akuntabilitas sistem. Dengan SET NULL, kolom id_pegawai pada baris log lama cukup dikosongkan menjadi NULL, sementara kolom aktivitas dan keterangan tetap utuh sebagai bukti historis yang valid.

Index dibuat pada waktu_aktivitas (untuk query audit berdasarkan rentang waktu) dan id_pegawai (untuk filter log per pegawai). Pengisian otomatis log aktivitas dilakukan oleh trigger trg_after_pembayaran_insert dan stored procedure sp_proses_pembayaran_aman serta sp_batal_reservasi_otomatis, sehingga pegawai tidak perlu mencatat log secara manual.

9. Trigger otomasi status kamar

```
-- AUTOMATION: TRIGGERS UNTUK OTOMASI STATUS KAMAR
-- =====
DELIMITER //

-- Trigger 1: Otomatis ubah status kamar menjadi 'Terisi' saat check-
in terjadi
CREATE TRIGGER trg_after_checkin
AFTER INSERT ON checkin
FOR EACH ROW
BEGIN
    UPDATE kamar
    SET status_kamar = 'Terisi'
    WHERE id_kamar IN (
        SELECT id_kamar FROM detail_reservasi WHERE id_reservasi =
NEW.id_reservasi
    );

    UPDATE reservasi
    SET status_reservasi = 'Check-in'
    WHERE id_reservasi = NEW.id_reservasi;
END //

-- Trigger 2: Otomatis ubah status kamar menjadi 'Tersedia' setelah
check-out selesai
CREATE TRIGGER trg_after_checkout
AFTER INSERT ON checkout
FOR EACH ROW
BEGIN
    UPDATE kamar
    SET status_kamar = 'Tersedia'
    WHERE id_kamar IN (
        SELECT id_kamar FROM detail_reservasi WHERE id_reservasi =
NEW.id_reservasi
    );

    UPDATE reservasi
    SET status_reservasi = 'Selesai'
    WHERE id_reservasi = NEW.id_reservasi;
END //
```

DELIMITER ;

Kedua trigger bekerja secara AFTER INSERT, dieksekusi setelah baris baru berhasil disimpan di tabel checkin atau checkout. Dengan mekanisme ini, pegawai cukup mencatat check-in/check-out, dan status kamar serta reservasi diperbarui secara otomatis oleh database tanpa intervensi manual.

No	Nama Tabel	Fungsi
1	kamar	Menyimpan data kamar fisik hotel.
2	reservasi	Menyimpan transaksi reservasi.
3	detail_reservasi	Menyimpan rincian kamar pada reservasi.
4	pembayaran	Menyimpan data pembayaran reservasi.
5	checkin	Menyimpan data realisasi check-in.
6	checkout	Menyimpan data realisasi check-out.
7	kamar_fasilitas	Menyimpan relasi many-to-many antara kamar dan fasilitas.
8	log_aktivitas	Menyimpan catatan aktivitas sistem.

4.3 Implementasi DML (Data Manipulation Language)

DML digunakan untuk mengisi basis data dengan data dummy yang realistis sesuai kebutuhan pengujian sistem. Data dimasukkan dengan urutan yang memperhatikan ketergantungan foreign key.

No	Nama Data	Contoh Jumlah Data	Catatan
1	INSERT TAMU	20 tamu	Tamu dari berbagai kota di Indonesia dengan data kontak nyata.
2	INSERT PEGAWAI	10 pegawai	Pegawai dengan 6 jabatan berbeda (Resepsionis, Kasir, Supervisor, dll.).
3	INSERT	5 tipe kamar	Standard (Rp350.000) hingga Suite (Rp1.250.000)

	TIPE_KAMAR		per malam.
4	INSERT FASILITAS	10 fasilitas	Wi-Fi, AC, TV LED, Mini Bar, Breakfast, Kolam Renang, Gym, Bathub, Room Service dan Laundry.
5	INSERT KAMAR	25 kamar	5 kamar per tipe, tersebar di lantai 1–5.
6	INSERT RESERVASI	22 reservasi	Mencakup seluruh status: Selesai, Check-in, Dikonfirmasi, Menunggu, Dibatalkan.
7	INSERT DETAIL_RESERVASI	26 detail reservasi	Beberapa reservasi memesan lebih dari satu kamar.
8	INSERT PEMBAYARAN	23 pembayaran	Termasuk pembayaran cicilan, refund, dan status pending.
9	INSERT CHECKIN dan INSERT CHECKOUT	10 check-in dan 12 check-out	10 reservasi selesai + 2 reservasi aktif yang sudah check-in. Seluruh reservasi yang berstatus Selesai.
10	INSERT KAMAR_FASILITAS	25 relasi kamar-fasilitas	Fasilitas bertambah sesuai tipe (Standard: 3 fasilitas, Suite: 10 fasilitas).
11	INSERT LOG_AKTIVITAS	12 log aktivitas	Mencatat aktivitas dari berbagai jabatan pegawai.

Di akhir script DML, disertakan query validasi untuk memastikan generated column berfungsi dengan benar:

```
SELECT
    dr.id_reservasi,
    t.nama_tamu,
    k.nomor_kamar,
    dr.jumlah_malam,
    dr.harga_per_malam,
    dr.subtotal AS 'Subtotal (Dihitung Otomatis)'
FROM detail_reservasi dr
JOIN reservasi r    ON dr.id_reservasi = r.id_reservasi
JOIN tamu t        ON r.id_tamu       = t.id_tamu
JOIN kamar k        ON dr.id_kamar     = k.id_kamar
LIMIT 5;
```

4.4 Constraint yang Digunakan

Jenis Constraint	Jumlah	Tabel / Keterangan
PRIMARY KEY	12	Satu PK per tabel; 1 composite PK (kamar_fasilitas).
FOREIGN KEY	14	Seluruh relasi antar tabel dijaga dengan FK.
UNIQUE	8	email (tamu, pegawai), no_identitas, nomor_kamar, nama_tipe, nama_fasilitas, id_reservasi (checkin, checkout), (id_reservasi, id_kamar).
CHECK	6	kapasitas > 0, harga_per_malam >= 0, jumlah_malam > 0, harga_per_malam (detail) >= 0, biaya_tambahan >= 0, checkout_date > checkin_date.
NOT NULL	Semua kolom wajib	Diterapkan pada semua kolom yang tidak boleh kosong.
DEFAULT	7	status_kamar, status_reservasi, status_pembayaran, is_active, biaya_tambahan, created_at, tanggal_reservasi.
GENERATED	1	subtotal pada detail_reservasi (STORED).
INDEX	7	idx_kamar_tipe, idx_kamar_status, idx_reservasi_tamu, idx_reservasi_pegawai, idx_reservasi_status, idx_log_waktu, idx_log_pegawai.
ON UPDATE CASCADE	8	Perubahan PK otomatis diteruskan ke FK terkait.
ON DELETE RESTRICT	6	Mencegah penghapusan data master yang masih direferensikan.
ON DELETE CASCADE	4	Penghapusan reservasi otomatis menghapus detail/payment/checkin/checkout.
ON DELETE SET NULL	1	log_aktivitas.id_pegawai → NULL jika pegawai dihapus.

4.5 Kesimpulan

Berdasarkan implementasi DDL dan DML, basis data hotel_reservation_db berhasil dibangun dengan 12 tabel utama yang saling terhubung melalui foreign key. Struktur tabel telah dilengkapi dengan constraint dan index untuk menjaga integritas data serta meningkatkan performa query. Data dummy yang realistis telah dimasukkan ke dalam seluruh tabel utama menunjukkan bahwa basis data mampu mendukung proses operasional hotel secara terintegrasi, mulai dari reservasi hingga check-out sehingga sistem siap digunakan untuk pengujian query, view, stored procedure, trigger, dan function.

BAB 5

PENGUJIAN QUERY

5.1 Query

1. Query Sederhana

a. Pemantauan kamar siap jual dan status perawatan real time

Tujuan: Membantu resepsionis melihat kamar yang dapat langsung ditawarkan atau sedang dalam perawatan.

```
SELECT
    k.nomor_kamar,
    k.lantai,
    tk.nama_tipe,
    CONCAT('Rp', FORMAT(tk.harga_per_malam, 0, 'id_ID')) AS
harga_per_malam,
    k.status_kamar
FROM kamar k
JOIN tipe_kamar tk ON k.id_tipe_kamar = tk.id_tipe_kamar
WHERE k.status_kamar IN ('Tersedia', 'Perawatan')
ORDER BY k.lantai ASC, k.nomor_kamar ASC;
```

Query ini melakukan JOIN antara tabel kamar dan tipe_kamar untuk mendapatkan nama tipe dan harga. Filter WHERE membatasi hasil hanya pada kamar dengan status Tersedia atau Perawatan. Fungsi FORMAT() memformat angka harga menjadi format Rupiah yang mudah dibaca.

b. Manifest checkin lengkap dengan detail tamu

Tujuan: Dokumen pegangan Front Office berisi daftar seluruh reservasi berikut kamar yang dipesan, kontak tamu, dan jadwal.

```
SELECT
    r.id_reservasi,
    t.nama_tamu,
    t.no_telepon,
    GROUP_CONCAT(k.nomor_kamar SEPARATOR ', ') AS
kamar_dipesan,
    DATE_FORMAT(r.tanggal_checkin_rencana, '%d-%m-%Y') AS
jadwal_checkin,
    DATE_FORMAT(r.tanggal_checkout_rencana, '%d-%m-%Y') AS
jadwal_checkout,
    r.status_reservasi
FROM reservasi r
JOIN tamu t ON r.id_tamu = t.id_tamu
JOIN detail_reservasi dr ON r.id_reservasi = dr.id_reservasi
JOIN kamar k ON dr.id_kamar = k.id_kamar
GROUP BY r.id_reservasi, t.nama_tamu, t.no_telepon,
         r.tanggal_checkin_rencana, r.tanggal_checkout_rencana,
         r.status_reservasi
ORDER BY r.tanggal_checkin_rencana DESC;
```

Melibatkan 4 tabel (reservasi, tamu, detail_reservasi, kamar).

GROUP_CONCAT menggabungkan nomor kamar menjadi satu string apabila satu reservasi memesan lebih dari satu kamar. DATE_FORMAT memformat tanggal ke notasi Indonesia.

c. Pemetaan fasilitas per kamar

Tujuan: Memberikan informasi akurat kepada tamu mengenai fasilitas yang tersedia di kamar mereka.

```
SELECT
    k.nomor_kamar,
    tk.nama_tipe,
    GROUP_CONCAT(f.nama_fasilitas SEPARATOR ' | ') AS
daftar_fasilitas
FROM kamar_fasilitas kf
JOIN kamar k ON kf.id_kamar = k.id_kamar
JOIN tipe_kamar tk ON k.id_tipe_kamar = tk.id_tipe_kamar
JOIN fasilitas f ON kf.id_fasilitas = f.id_fasilitas
GROUP BY k.nomor_kamar, tk.nama_tipe
ORDER BY k.nomor_kamar ASC;
```

Memanfaatkan tabel junction kamar_fasilitas yang menghubungkan relasi many-to-many antara kamar dan fasilitas. GROUP_CONCAT mengagregasi semua fasilitas per kamar menjadi satu baris output.

2. Query dengan JOIN

a. Laporan invoice/total tagihan akhir tamu

Tujuan: Menghitung total tagihan lengkap (biaya kamar + biaya tambahan checkout) untuk seluruh reservasi. Melibatkan 6 tabel: reservasi, tamu, detail_reservasi, kamar, tipe_kamar, checkin, checkout.

```
SELECT
    r.id_reservasi AS 'ID Reservasi',
    t.nama_tamu AS 'Nama Tamu',
    GROUP_CONCAT(DISTINCT k.nomor_kamar SEPARATOR ', ') AS 'Nomor
Kamar',
    GROUP_CONCAT(DISTINCT tk.nama_tipe SEPARATOR ', ') AS 'Tipe
Kamar',
    SUM(dr.jumlah_malam) AS 'Total Malam',
    COUNT(dr.id_kamar) AS 'Jumlah Kamar',
    DATE_FORMAT(ci.waktu_checkin, '%d-%m-%Y %H:%i') AS 'Waktu
Check-In',
    DATE_FORMAT(co.waktu_checkout, '%d-%m-%Y %H:%i') AS 'Waktu
Check-Out',
    CONCAT('Rp', FORMAT(SUM(dr.subtotal), 0, 'id_ID'))
    AS 'Total Biaya Kamar',
    CONCAT('Rp', FORMAT(COALESCE(co.biaya_tambahan, 0), 0,
'id_ID'))
    AS 'Biaya Tambahan',
    CONCAT('Rp', FORMAT(SUM(dr.subtotal)
    + COALESCE(co.biaya_tambahan, 0), 0, 'id_ID'))
    AS 'GRAND TOTAL'
```

```

FROM reservasi r
JOIN tamu t ON r.id_tamu = t.id_tamu
JOIN detail_reservasi dr ON r.id_reservasi = dr.id_reservasi
JOIN kamar k ON dr.id_kamar = k.id_kamar
JOIN tipe_kamar tk ON k.id_tipe_kamar = tk.id_tipe_kamar
LEFT JOIN checkin ci ON r.id_reservasi = ci.id_reservasi
LEFT JOIN checkout co ON r.id_reservasi = co.id_reservasi
GROUP BY r.id_reservasi, t.nama_tamu,
         ci.waktu_checkin, co.waktu_checkout, co.biaya_tambahan
ORDER BY r.id_reservasi ASC;

```

Menggunakan LEFT JOIN pada checkin dan checkout agar reservasi yang belum check-in/out tetap muncul (dengan nilai NULL yang ditangani oleh COALESCE).

GRAND TOTAL merupakan penjumlahan subtotal kamar dan biaya tambahan.

3. Query dengan subquery/CTE

a. Laporan tren pendapatan bulanan (CTE)

Tujuan: Laporan bulanan untuk Direksi yang merangkum total omset dan jumlah transaksi pembayaran yang berhasil setiap bulan.

```

WITH PendapatanBulanan AS (
    SELECT
        DATE_FORMAT(tanggal_pembayaran, '%Y-%m') AS periode,
        SUM(jumlah_bayar) AS total_omset,
        COUNT(id_pembayaran) AS total_transaksi
    FROM pembayaran
    WHERE status_pembayaran = 'Lunas'
    GROUP BY DATE_FORMAT(tanggal_pembayaran, '%Y-%m')
)
SELECT
    periode,
    total_transaksi,
    CONCAT('Rp', FORMAT(total_omset, 0, 'id_ID'))
        AS omset_bersih_bulanan
FROM PendapatanBulanan
ORDER BY periode DESC;

```

CTE (Common Table Expression) PendapatanBulanan mengelompokkan pembayaran berhasil per bulan menggunakan DATE_FORMAT. Query utama kemudian membaca hasil CTE untuk ditampilkan dengan format Rupiah. Pendekatan CTE meningkatkan keterbacaan dibandingkan subquery bersarang.

b. Analisis kamar terlaris

Tujuan: Strategi marketing untuk mengetahui tipe kamar mana yang paling diminati pasar.

```

SELECT
    tk.nama_tipe,
    COUNT(dr.id_kamar) AS total_kali_dipesan,
    SUM(dr.jumlah_malam) AS akumulasi_malam_terjual,
    CONCAT('Rp', FORMAT(SUM(dr.subtotal), 0, 'id_ID')) AS
    kontribusi_pendapatan

```

```
FROM detail_reservasi dr
JOIN kamar k
    ON dr.id_kamar = k.id_kamar
JOIN tipe_kamar tk
    ON k.id_tipe_kamar = tk.id_tipe_kamar
GROUP BY tk.id_tipe_kamar, tk.nama_tipe
ORDER BY total_kali_dipesan DESC;
```

Penjelasan: Query melibatkan tiga tabel — detail_reservasi, kamar, dan tipe_kamar — karena tidak ada foreign key langsung dari detail reservasi ke tipe kamar, sehingga harus melewati tabel kamar sebagai perantara. Tiga fungsi agregat digunakan sekaligus: COUNT untuk volume pemesanan, SUM(jumlah_malam) untuk total durasi, dan SUM(subtotal) untuk total pendapatan per tipe. Hasil diurutkan DESC sehingga tipe kamar terlaris muncul di baris pertama.

c. Deteksi tamu loyal/pelanggan setia (subquery implisit via HAVING)

Tujuan: Mengidentifikasi tamu yang telah melakukan minimal 2 reservasi sebagai kandidat program loyalitas atau diskon member.

```
SELECT
    t.id_tamu,
    t.nama_tamu,
    t.email,
    COUNT(r.id_reservasi) AS total_kunjungan,
    GROUP_CONCAT(DISTINCT r.status_reservasi
                  SEPARATOR ', ') AS histori_status
FROM tamu t
JOIN reservasi r ON t.id_tamu = r.id_tamu
GROUP BY t.id_tamu, t.nama_tamu, t.email
HAVING total_kunjungan >= 2
ORDER BY total_kunjungan DESC;
```

Penjelasan: HAVING memfilter hasil agregasi (berbeda dengan WHERE yang memfilter baris sebelum agregasi). GROUP_CONCAT DISTINCT menampilkan variasi status reservasi dari tamu tersebut selama riwayat menginapnya.

5.2 View

Terdapat 5 view yang diimplementasikan dalam file 04_view.sql. Dua view utama dideskripsikan secara rinci berikut ini, sisanya dijelaskan dalam ringkasan

1. View dashboard real time reservasi tamu

Tujuan: Dashboard real-time yang menampilkan detail lengkap setiap reservasi berikut informasi tamu, kamar, tipe, harga, dan status. Digunakan oleh resepsionis sebagai tampilan utama sistem.

View ini menggabungkan 5 tabel sekaligus sehingga query di lapisan aplikasi dapat sesederhana SELECT * FROM vw_detail_reservasi_tamu. Tanggal diformat ke notasi

Indonesia menggunakan DATE_FORMAT.

2. View billing reservasi summary

Tujuan: Rekapitulasi billing per reservasi yang menghitung total biaya kamar, biaya tambahan checkout, grand total tagihan, total yang sudah dibayar, dan sisa tagihan secara real-time. Digunakan oleh kasir untuk monitoring piutang.

Menggunakan correlated subquery pada kolom total_telah_dibayar untuk menghitung akumulasi pembayaran lunas per reservasi. LEFT JOIN pada checkout memastikan reservasi yang belum checkout tetap muncul dengan biaya tambahan = 0 (ditangani IFNULL).

Ringkasan 3 View Tambahan:

View	Tujuan	Pengguna
vw_laporan_pembayaran	Riwayat seluruh transaksi pembayaran dengan informasi tamu. Format datetime lengkap sampai detik.	Kasir / Finance
vw_status_kamar_opsional	Peta status seluruh kamar real-time berikut nama tamu yang sedang menempati (CASE WHEN subquery). Tampilan dashboard housekeeping.	Resepsionis / Housekeeping
vw_performa_staf_operasional	KPI produktivitas pegawai: jumlah reservasi yang ditangani, jumlah check-in, dan jumlah check-out per pegawai aktif.	Manager / HRD

```
USE hotel_reservation_db;

-- =====
-- 1. VIEW: DASHBOARD REAL-TIME RESERVASI TAMU
-- Fungsi: Menampilkan detail reservasi aktif/lalu secara lengkap.
-- Upgrade: Menambahkan format mata uang lokal (Rupiah) dan status
pembayaran summary.
-- =====
CREATE OR REPLACE VIEW vw_detail_reservasi_tamu AS
SELECT
    r.id_reservasi,
    t.nama_tamu,
    t.no_telepon,
    k.nomor_kamar,
    tk.nama_tipe,
    -- Memformat tanggal agar lebih mudah dibaca oleh UI Aplikasi
    DATE_FORMAT(r.tanggal_reservasi, '%d-%m-%Y %H:%i') AS
tanggal_booking,
    DATE_FORMAT(r.tanggal_checkin_rencana, '%d-%m-%Y') AS
rencana_checkin,
    DATE_FORMAT(r.tanggal_checkout_rencana, '%d-%m-%Y') AS
rencana_checkout,
    dr.jumlah_malam,
    dr.harga_per_malam,
```

```

        dr.subtotal,
        r.status_reservasi
FROM reservasi r
JOIN tamu t ON r.id_tamu = t.id_tamu
JOIN detail_reservasi dr ON r.id_reservasi = dr.id_reservasi
JOIN kamar k ON dr.id_kamar = k.id_kamar
JOIN tipe_kamar tk ON k.id_tipe_kamar = tk.id_tipe_kamar;

-- =====
-- 2. VIEW: LAPORAN KEUANGAN & RIWAYAT PEMBAYARAN
-- Fungsi: Digunakan oleh Kasir/Finance untuk audit transaksi masuk.
-- Upgrade: Menggunakan DATETIME (sesuai DDL baru) dan mengurutkan
secara kronologis.
-- =====
CREATE OR REPLACE VIEW vw_laporan_pembayaran AS
SELECT
    pb.id_pembayaran,
    r.id_reservasi,
    t.nama_tamu,
    DATE_FORMAT(pb.tanggal_pembayaran, '%d-%m-%Y %H:%i:%s') AS
waktu_pembayaran,
    pb.jumlah_bayar,
    pb.metode_pembayaran,
    pb.status_pembayaran
FROM pembayaran pb
JOIN reservasi r ON pb.id_reservasi = r.id_reservasi
JOIN tamu t ON r.id_tamu = t.id_tamu;

-- =====
-- 3. UPGRADE: VIEW REKAPITULASI BILL/BILLING UTAMA (BARU)
-- Fungsi: Menghitung total biaya kamar vs total yang sudah dibayar
secara realtime per reservasi.
-- Kegunaan: Mengetahui secara instan apakah tamu "Kurang Bayar" atau
"Lunas".
-- =====
CREATE OR REPLACE VIEW vw_billing_reservasi_summary AS
SELECT
    r.id_reservasi,
    t.id_tamu,
    t.nama_tamu,
    r.status_reservasi,
    -- Menghitung total biaya seluruh kamar yang dipesan di reservasi
tersebut
    IFNULL(SUM(dr.subtotal), 0) AS total_biaya_kamar,
    -- Mengambil biaya tambahan saat checkout jika ada
    IFNULL(co.biaya_tambahan, 0) AS biaya_tambahan_checkout,
    -- Total keseluruhan yang harus dibayar
    (IFNULL(SUM(dr.subtotal), 0) + IFNULL(co.biaya_tambahan, 0)) AS
grand_total_tagihan,
    -- Menghitung total uang yang sudah dibayar (mengantisipasi split
payment)
    IFNULL((SELECT SUM(jumlah_bayar) FROM pembayaran WHERE
id_reservasi = r.id_reservasi AND status_pembayaran = 'Lunas'), 0) AS
total_telah_dibayar,
    -- Menghitung sisa tagihan secara matematis
    ((IFNULL(SUM(dr.subtotal), 0) + IFNULL(co.biaya_tambahan, 0)) -
    IFNULL((SELECT SUM(jumlah_bayar) FROM pembayaran WHERE
id_reservasi = r.id_reservasi AND status_pembayaran = 'Lunas'), 0))
AS sisa_tagihan
FROM reservasi r

```

```

JOIN tamu t ON r.id_tamu = t.id_tamu
LEFT JOIN detail_reservasi dr ON r.id_reservasi = dr.id_reservasi
LEFT JOIN checkout co ON r.id_reservasi = co.id_reservasi
GROUP BY r.id_reservasi, t.id_tamu, co.biaya_tambahan;

-- =====
-- 4. UPGRADE: VIEW OKUPANSI & STATUS KAMAR REAL-TIME (BARU)
-- Fungsi: Keperluan resepsionis (Front Office) saat mau memetakan
-- kamar kosong/isi.
-- =====
CREATE OR REPLACE VIEW vw_status_kamar_opsional AS
SELECT
    k.id_kamar,
    k.nomor_kamar,
    k.lantai,
    tk.nama_tipe,
    tk.harga_per_malam,
    k.status_kamar,
    -- Menampilkan info tamu yang sedang menempati kamar saat ini
    (jika ada)
    CASE
        WHEN k.status_kamar = 'Terisi' THEN (
            SELECT t.nama_tamu
            FROM detail_reservasi dr
            JOIN reservasi r ON dr.id_reservasi = r.id_reservasi
            JOIN tamu t ON r.id_tamu = t.id_tamu
            WHERE dr.id_kamar = k.id_kamar AND r.status_reservasi =
'Check-in'
            LIMIT 1
        )
        ELSE '-'
    END AS nama_tamu_sekarang
FROM kamar k
JOIN tipe_kamar tk ON k.id_tipe_kamar = tk.id_tipe_kamar;

-- =====
-- 5. UPGRADE: VIEW KINERJA PRODUKTIVITAS PEGAWAI (BARU)
-- Fungsi: Keperluan manajemen/HRD untuk melihat performa kerja staf
-- (KPI).
-- =====
CREATE OR REPLACE VIEW vw_performa_staf_operasional AS
SELECT
    p.id_pegawai,
    p.nama_pegawai,
    p.jabatan,
    (SELECT COUNT(*) FROM reservasi WHERE id_pegawai = p.id_pegawai)
AS jumlah_handle_reservasi,
    (SELECT COUNT(*) FROM checkin WHERE id_pegawai = p.id_pegawai) AS
jumlah_handle_checkin,
    (SELECT COUNT(*) FROM checkout WHERE id_pegawai = p.id_pegawai)
AS jumlah_handle_checkout
FROM pegawai p
WHERE p.is_active = 1;

```

5.3 Stored Procedure

1. Stored procedure menghitung invoice lengkap

Tujuan: Menghitung rincian finansial lengkap sebuah reservasi dan

mengembalikan hasilnya melalui parameter OUT. Prosedur ini menjadi komponen dasar yang dipanggil ulang (reusability) oleh prosedur lain.

```
CREATE PROCEDURE sp_hitung_invoice_lengkap(
    IN p_id_reservasi INT,
    OUT p_biaya_kamar DECIMAL(12,2),
    OUT p_biaya_tambahan DECIMAL(12,2),
    OUT p_grand_total DECIMAL(12,2),
    OUT p_total_dibayar DECIMAL(12,2),
    OUT p_sisa_tagihan DECIMAL(12,2)
)
BEGIN
    SELECT COALESCE(SUM(subtotal), 0) INTO p_biaya_kamar
    FROM detail_reservasi WHERE id_reservasi = p_id_reservasi;

    SELECT COALESCE(biaya_tambahan, 0) INTO p_biaya_tambahan
    FROM checkout WHERE id_reservasi = p_id_reservasi;

    SET p_grand_total = p_biaya_kamar + p_biaya_tambahan;

    SELECT COALESCE(SUM(jumlah_bayar), 0) INTO p_total_dibayar
    FROM pembayaran
    WHERE id_reservasi = p_id_reservasi
    AND status_pembayaran = 'Lunas';

    SET p_sisa_tagihan = p_grand_total - p_total_dibayar;
END;

-- Cara pemanggilan:
CALL sp_hitung_invoice_lengkap(1, @kamar, @tambahan,
    @total, @dibayar, @sisas);
SELECT @kamar, @tambahan, @total, @dibayar, @sisas;
```

2. Stored procedure pembayaran aman

Tujuan: Mencatat pembayaran baru secara aman dengan validasi sisa tagihan, transaksi ACID, dan pencatatan otomatis ke log aktivitas. Mendukung skenario cicilan (partial payment).

```
CREATE PROCEDURE sp_proses_pembayaran_aman(
    IN p_id_reservasi INT,
    IN p_jumlah_bayar DECIMAL(12,2),
    IN p_metode ENUM('Tunai', 'Transfer',
        'Kartu Kredit', 'E-Wallet'),
    IN p_id_pegawai INT,
    OUT p_status_pesan VARCHAR(100)
)
BEGIN
    -- Ambil data invoice terkini (memanggil procedure 1)
    CALL sp_hitung_invoice_lengkap(p_id_reservasi,
        v_biaya_kamar, v_biaya_tambahan,
        v_grand_total, v_total_dibayar, v_sisa_tagihan);

    IF v_sisa_tagihan <= 0 THEN
        SET p_status_pesan = 'GAGAL: Tagihan sudah lunas.';
    ELSE
        START TRANSACTION;
        INSERT INTO pembayaran (...) VALUES (...);
```

```

        UPDATE reservasi SET status_reservasi = 'Dikonfirmasi'
        WHERE id_reservasi = p_id_reservasi
        AND status_reservasi = 'Menunggu'
        AND v_sisa_tagihan <= 0;
        INSERT INTO log_aktivitas (...) VALUES (...);
    COMMIT;
    SET p_status_pesan = CONCAT('SUKSES: Sisa tagihan: Rp',
        FORMAT(v_sisa_tagihan, 0, 'id_ID'));
END IF;
END;

-- Cara pemanggilan:
CALL sp_proses_pembayaran_aman(19, 500000.00,
    'Transfer', 3, @pesan);
SELECT @pesan;

```

Fitur kunci: (1) Validasi sisa tagihan sebelum mencatat pembayaran. (2) Menggunakan START TRANSACTION / COMMIT untuk menjamin atomisitas operasi (prinsip ACID). (3) Memanggil sp_hitung_invoice_lengkap sebagai sub-komponen (reusability antar prosedur).

3. Stored procedure batal reservasi otomatis

Tujuan: Membatalkan reservasi secara aman dengan satu perintah — mengubah status reservasi, membebaskan kamar, mengeset pembayaran ke Refund, dan mencatat log pembatalan secara atomik.

```

CREATE PROCEDURE sp_batal_reservasi_otomatis(
    IN p_id_reservasi INT,
    IN p_id_pegawai INT,
    OUT p_status_pesan VARCHAR(100)
)
BEGIN
    SELECT status_reservasi INTO v_status_sekarang
    FROM reservasi WHERE id_reservasi = p_id_reservasi;

    -- Blokir pembatalan bila sudah Check-in/Selesai/Dibatalkan
    IF v_status_sekarang IN ('Check-in', 'Selesai', 'Dibatalkan') THEN
        SET p_status_pesan = CONCAT('GAGAL: Status sudah ',
            v_status_sekarang);
    ELSE
        START TRANSACTION;
        UPDATE reservasi SET status_reservasi = 'Dibatalkan'
        WHERE id_reservasi = p_id_reservasi;
        UPDATE kamar SET status_kamar = 'Tersedia'
        WHERE id_kamar IN (
            SELECT id_kamar FROM detail_reservasi
            WHERE id_reservasi = p_id_reservasi);
        UPDATE pembayaran SET status_pembayaran = 'Refund'
        WHERE id_reservasi = p_id_reservasi;
        INSERT INTO log_aktivitas (...) VALUES (...);
        COMMIT;
        SET p_status_pesan = 'SUKSES: Reservasi dibatalkan.';
    END IF;
END;

-- Cara pemanggilan:

```

```
CALL sp_batal_reservasi_otomatis(20, 6, @pesan);
SELECT @pesan;
```

Fitur kunci: Validasi guard clause mencegah pembatalan reservasi yang sudah berjalan.

Tiga operasi UPDATE (reservasi, kamar, pembayaran) dibungkus dalam satu transaksi sehingga jika salah satu gagal, seluruh perubahan dibatalkan.

5.4 Trigger

No	Nama Trigger	Event	Tabel	Fungsi Utama
1	trg_before_detail_reservasi_insert	BEFORE INSERT	detail_reservasi	Menolak pemesanan kamar yang sedang Perawatan dan mencegah double booking (irisan tanggal).
2	trg_after_detail_reservasi_insert	AFTER INSERT	detail_reservasi	Otomatis mengubah status kamar dari Tersedia menjadi Dipesan setelah plot kamar berhasil.
3	trg_after_checkin_insert	AFTER INSERT	checkin	Mengubah status kamar menjadi Terisi dan status reservasi menjadi Check-in.
4	trg_after_checkout_insert	AFTER INSERT	checkout	Mengembalikan status kamar ke Tersedia dan menutup reservasi dengan status Selesai.
5	trg_after_pembayaran_insert	AFTER INSERT	pembayaran	Mencatat audit trail otomatis ke log_aktivitas setiap kali ada pembayaran masuk.

1. trg_before_detail_reservasi_insert (Anti-Double Booking)

Trigger ini adalah proteksi inti sistem. Sebelum baris baru dimasukkan ke detail_reservasi, trigger memeriksa dua kondisi:

- Kondisi 1: Apakah status kamar = Perawatan? Jika ya, SIGNAL SQLSTATE menolak operasi.
- Kondisi 2: Apakah terdapat reservasi lain (yang belum Selesai/Dibatalkan) pada kamar yang sama dengan tanggal yang bertabrakan? Deteksi irisan menggunakan logika: $\text{checkin_lain} < \text{checkout_baru}$ AND $\text{checkout_lain} > \text{checkin_baru}$.

```
-- Logika deteksi irisan tanggal (double booking):
SELECT COUNT(*) INTO v_jumlah_bentrok
FROM detail_reservasi dr
JOIN reservasi r ON dr.id_reservasi = r.id_reservasi
WHERE dr.id_kamar = NEW.id_kamar
  AND dr.id_reservasi <> NEW.id_reservasi
  AND r.status_reservasi NOT IN ('Dibatalkan', 'Selesai')
  AND r.tanggal_checkin_rencana < v_tanggal_checkout
```

```

AND r.tanggal_checkout_rencana > v_tanggal_checkin;

IF v_jumlah_bentrok > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'OPERASI DITOLAK: Kamar sudah
    ter-booking pada periode tersebut.';
END IF;

```

Skenario Pengujian Trigger (dari 06_trigger.sql):

Test	Skenario	Ekspektasi
TEST 1	Insert ke kamar ID 5 (status Perawatan).	GAGAL — Error 1644: Kamar dalam masa pemeliharaan.
TEST 2	Insert kamar ID 1 dengan tanggal 2026-05-11 s.d. 2026-05-13 (bertabrakan dengan reservasi ID 1).	GAGAL — Error 1644: Double booking terdeteksi.
TEST 3	Insert kamar ID 4 (status Tersedia) tanggal 2026-08-01 s.d. 2026-08-03.	SUKSES — Status kamar berubah otomatis ke Dipesan (Trigger 2).
TEST 4	Insert pembayaran untuk reservasi ID 913.	SUKSES — Log aktivitas otomatis tercatat (Trigger 5).
TEST 5	Insert check-in untuk reservasi ID 913.	SUKSES — Status kamar → Terisi, status reservasi → Check-in (Trigger 3).
TEST 6	Insert check-out untuk reservasi ID 913.	SUKSES — Status kamar → Tersedia, status reservasi → Selesai (Trigger 4).

```

USE hotel_reservation_db;

--
=====
-- SECTION 1: DETEKSI & RE-INITIALIZATION TRIGGER
--
=====
DELIMITER //

DROP TRIGGER IF EXISTS trg_before_detail_reservasi_insert //
DROP TRIGGER IF EXISTS trg_after_detail_reservasi_insert //
DROP TRIGGER IF EXISTS trg_after_checkin_insert //
DROP TRIGGER IF EXISTS trg_after_checkout_insert //
DROP TRIGGER IF EXISTS trg_after_pembayaran_insert //

--
-----
-- TRIGGER 1: BEFORE INSERT ON detail_reservasi
-- Logika Bisnis: Menolak plot kamar jika berstatus 'Perawatan' ATAU
jika tanggal
--                hunian bertabrakan dengan reservasi aktif lain
(Anti Double Booking).
--
-----

```

```

CREATE TRIGGER trg_before_detail_reservasi_insert
BEFORE INSERT ON detail_reservasi
FOR EACH ROW
BEGIN
    DECLARE v_status_kamar      VARCHAR(20);
    DECLARE v_tanggal_checkin   DATE;
    DECLARE v_tanggal_checkout DATE;
    DECLARE v_jumlah_bentrok    INT DEFAULT 0;

    -- [A] Ambil status riil kamar fisik saat ini
    SELECT status_kamar INTO v_status_kamar
    FROM kamar
    WHERE id_kamar = NEW.id_kamar;

    IF v_status_kamar = 'Perawatan' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'OPERASI DITOLAK: Kamar tidak dapat
dipesan karena sedang dalam masa pemeliharaan/Perawatan.';
    END IF;

    -- [B] Ambil batas rentang tanggal hunian dari tabel header
(reservasi)
    SELECT tanggal_checkin_rencana, tanggal_checkout_rencana
    INTO v_tanggal_checkin, v_tanggal_checkout
    FROM reservasi
    WHERE id_reservasi = NEW.id_reservasi;

    -- [C] Kalkulasi irisan tanggal untuk mencegah terjadinya double
booking
    SELECT COUNT(*) INTO v_jumlah_bentrok
    FROM detail_reservasi dr
    JOIN reservasi r ON dr.id_reservasi = r.id_reservasi
    WHERE dr.id_kamar = NEW.id_kamar
        AND dr.id_reservasi <> NEW.id_reservasi
        AND r.status_reservasi NOT IN ('Dibatalkan', 'Selesai')
        AND r.tanggal_checkin_rencana < v_tanggal_checkout
        AND r.tanggal_checkout_rencana > v_tanggal_checkin;

    IF v_jumlah_bentrok > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'OPERASI DITOLAK: Kamar sudah ter-
booking oleh tamu lain pada periode tanggal tersebut.';
    END IF;
END //

-- -----
--
-- TRIGGER 2: AFTER INSERT ON detail_reservasi
-- Logika Bisnis: Mengubah kamar menjadi 'Dipesan' setelah plot
detail valid.
-- -----
--
CREATE TRIGGER trg_after_detail_reservasi_insert
AFTER INSERT ON detail_reservasi
FOR EACH ROW
BEGIN
    UPDATE kamar
    SET status_kamar = 'Dipesan'
    WHERE id_kamar = NEW.id_kamar
        AND status_kamar = 'Tersedia';
END //

```

```

-- -----
--
-- TRIGGER 3: AFTER INSERT ON checkin
-- Logika Bisnis: Mengubah kamar menjadi 'Terisi' dan status
reservasi menjadi 'Check-in'.
-- -----
--
CREATE TRIGGER trg_after_checkin_insert
AFTER INSERT ON checkin
FOR EACH ROW
BEGIN
    UPDATE kamar k
    JOIN detail_reservasi dr ON k.id_kamar = dr.id_kamar
    SET k.status_kamar = 'Terisi'
    WHERE dr.id_reservasi = NEW.id_reservasi;

    UPDATE reservasi
    SET status_reservasi = 'Check-in'
    WHERE id_reservasi = NEW.id_reservasi;
END //

-- -----
--
-- TRIGGER 4: AFTER INSERT ON checkout
-- Logika Bisnis: Mengembalikan kamar ke status 'Tersedia' & tutup
reservasi jadi 'Selesai'.
-- -----
--
CREATE TRIGGER trg_after_checkout_insert
AFTER INSERT ON checkout
FOR EACH ROW
BEGIN
    UPDATE kamar k
    JOIN detail_reservasi dr ON k.id_kamar = dr.id_kamar
    SET k.status_kamar = 'Tersedia'
    WHERE dr.id_reservasi = NEW.id_reservasi
    AND k.status_kamar <> 'Perawatan';

    UPDATE reservasi
    SET status_reservasi = 'Selesai'
    WHERE id_reservasi = NEW.id_reservasi;
END //

-- -----
--
-- TRIGGER 5: AFTER INSERT ON pembayaran
-- Logika Bisnis: Audit Trail otomatis yang merekam mutasi uang masuk
ke log_aktivitas.
-- -----
--
CREATE TRIGGER trg_after_pembayaran_insert
AFTER INSERT ON pembayaran
FOR EACH ROW
BEGIN
    INSERT INTO log_aktivitas (id_pegawai, aktivitas, keterangan)
    SELECT
        r.id_pegawai,
        'Pembayaran Masuk',
        CONCAT('Pembayaran Sukses untuk Reservasi ID ',
NEW.id_reservasi,

```

```

        ' sebesar Rp', FORMAT(NEW.jumlah_bayar, 0, 'id_ID'),
        ' melalui metode [' , NEW.metode_pembayaran, '].')
    FROM reservasi r
    WHERE r.id_reservasi = NEW.id_reservasi;
END //

DELIMITER ;

--
=====
-- SECTION 2: AUTOMATED TESTING SCRIPTS (INTEGRATION VALIDATION)
--
=====

-- Skenario 0: Pembersihan Total Data Pengujian Sebelumnya (Mencegah
Duplicate Key)
SET SQL_SAFE_UPDATES = 0;

DELETE FROM log_aktivitas WHERE keterangan LIKE '%Reservasi ID 91%';
DELETE FROM checkout WHERE id_reservasi IN (911, 912, 913);
DELETE FROM checkin WHERE id_reservasi IN (911, 912, 913);
DELETE FROM pembayaran WHERE id_reservasi IN (911, 912, 913);
DELETE FROM detail_reservasi WHERE id_reservasi IN (911, 912, 913);
DELETE FROM reservasi WHERE id_reservasi IN (911, 912, 913);

-- Kembalikan status kamar master uji coba ke kondisi default sistem
UPDATE kamar SET status_kamar = 'Tersedia' WHERE id_kamar IN (1, 4);
UPDATE kamar SET status_kamar = 'Perawatan' WHERE id_kamar = 5;

SET SQL_SAFE_UPDATES = 1;

-----
-----
-- KELOMPOK A: PENGUJIAN SKENARIO NEGATIF (UJI COBA PROTEKSI TRIGGER)
-- Catatan Profesional: Blok kode di bawah ini memicu pesan
pembatasan data secara sengaja.
-----
-----

-- [TEST 1] Proteksi Kamar Rusak/Dalam Pemeliharaan
-- Ekspektasi: Langkah 1 Sukses masuk, Langkah 2 WAJIB GAGAL
memunculkan Error 1644 (Kamar Perawatan)
INSERT INTO reservasi (id_reservasi, id_tamu, id_pegawai,
tanggal_reservasi, tanggal_checkin_rencana, tanggal_checkout_rencana,
status_reservasi)
VALUES (911, 3, 2, NOW(), '2026-07-01', '2026-07-03', 'Menunggu');

-- Eksekusi Detail (Akan digagalkan oleh Trigger 1)
INSERT INTO detail_reservasi (id_detail_reservasi, id_reservasi,
id_kamar, jumlah_malam, harga_per_malam)
VALUES (911, 911, 5, 2, 350000.00);

-- [TEST 2] Proteksi Double Booking (Tabrakan Jadwal Kamar)
-- Ekspektasi: Langkah 1 Sukses masuk, Langkah 2 WAJIB GAGAL
memunculkan Error 1644 (Tabrakan Tanggal dengan Reservasi ID 1)
INSERT INTO reservasi (id_reservasi, id_tamu, id_pegawai,

```

```

tanggal_reservasi, tanggal_checkin_rencana, tanggal_checkout_rencana,
status_reservasi)
VALUES (912, 1, 1, NOW(), '2026-05-11', '2026-05-13',
'Dikonfirmasi');

-- Eksekusi Detail (Akan digagalkan oleh Trigger 1)
INSERT INTO detail_reservasi (id_detail_reservasi, id_reservasi,
id_kamar, jumlah_malam, harga_per_malam)
VALUES (912, 912, 1, 2, 350000.00);

-----
-----
-- KELOMPOK B: PENGUJIAN SKENARIO POSITIF (ALUR SIKLUS SUKSES END-TO-
END)
-- Catatan Profesional: Blok kode di bawah ini dirancang berjalan
linier tanpa error.
-----
-----

-- [TEST 3] Alur Pemesanan (Booking) Kamar yang Tersedia
-- Ekspektasi: Data masuk sempurna dan status kamar otomatis berubah
menjadi 'Dipesan'.
INSERT INTO reservasi (id_reservasi, id_tamu, id_pegawai,
tanggal_reservasi, tanggal_checkin_rencana, tanggal_checkout_rencana,
status_reservasi)
VALUES (913, 2, 4, NOW(), '2026-08-01', '2026-08-03',
'Dikonfirmasi');

INSERT INTO detail_reservasi (id_detail_reservasi, id_reservasi,
id_kamar, jumlah_malam, harga_per_malam)
VALUES (913, 913, 4, 2, 350000.00);

-- ASSERT CHECK 3: Memastikan Kamar 104 berubah status menjadi
'Dipesan' (Efek Trigger 2)
SELECT nomor_kamar, status_kamar FROM kamar WHERE id_kamar = 4;

-- [TEST 4] Simulasi Pembayaran Tagihan Masuk
-- Ekspektasi: Transaksi keuangan tersimpan dan log aktivitas terbuat
otomatis dengan rapi.
INSERT INTO pembayaran (id_reservasi, jumlah_bayar,
metode_pembayaran, status_pembayaran)
VALUES (913, 700000.00, 'E-Wallet', 'Lunas');

-- ASSERT CHECK 4: Memeriksa rekaman teks audit finansial format
Rupiah hasil olahan Trigger 5
SELECT keterangan, waktu_aktivitas FROM log_aktivitas ORDER BY id_log
DESC LIMIT 1;

-- [TEST 5] Simulasi Kedatangan Tamu (Proses Check-In)
-- Ekspektasi: Status kamar fisik berubah menjadi 'Terisi' & Status
master reservasi berubah menjadi 'Check-in'.
INSERT INTO checkin (id_checkin, id_reservasi, id_pegawai, catatan)
VALUES (913, 913, 4, 'Tamu melakukan check-in, kamar siap
digunakan.');
```

```

-- ASSERT CHECK 5: Memastikan sinkronisasi status kamar dan data
transaksi (Efek Trigger 3)
SELECT nomor_kamar, status_kamar FROM kamar WHERE id_kamar = 4;
```

```

SELECT id_reservasi, status_reservasi FROM reservasi WHERE
id_reservasi = 913;

-- [TEST 6] Simulasi Tamu Pulang (Proses Check-Out Selesai)
-- Ekspektasi: Kamar otomatis dikosongkan kembali ('Tersedia') &
Status master reservasi ditutup dengan status 'Selesai'.
INSERT INTO checkout (id_checkout, id_reservasi, id_pegawai,
biaya_tambahan, catatan)
VALUES (913, 913, 2, 0.00, 'Check-out selesai tanpa ada kehilangan
fasilitas.');
```

```

-- ASSERT CHECK 6: Memastikan kamar telah bersih dan siap disewa
kembali oleh pelanggan baru (Efek Trigger 4)
SELECT nomor_kamar, status_kamar FROM kamar WHERE id_kamar = 4;
SELECT id_reservasi, status_reservasi FROM reservasi WHERE
id_reservasi = 913;
```

5.5 Stored Function

1. Function hitung durasi malam

Tujuan: Menghitung selisih malam antara tanggal check-in dan check-out dengan validasi input.

```

CREATE FUNCTION fn_hitung_durasi_malam(
    p_checkin DATE,
    p_checkout DATE
)
RETURNS INT
DETERMINISTIC
BEGIN
    -- Validasi: tanggal tidak logis → kembalikan 0
    IF p_checkout <= p_checkin
        OR p_checkin IS NULL
        OR p_checkout IS NULL THEN
        RETURN 0;
    END IF;
    RETURN DATEDIFF(p_checkout, p_checkin);
END;
```

```

-- Contoh penggunaan:
SELECT fn_hitung_durasi_malam('2026-06-01', '2026-06-05');
```

-- Output: 4

Atribut DETERMINISTIC menandakan fungsi selalu menghasilkan output yang sama untuk input yang sama digunakan oleh query optimizer MySQL untuk meningkatkan performa.

2. Function total pendapatan reservasi

Tujuan: Menghitung total piutang kotor (biaya kamar + biaya tambahan checkout) untuk satu reservasi. Digunakan sebagai komponen dasar oleh fn_cek_status_pembayaran.

```

CREATE FUNCTION fn_total_pendapatan_reservasi(
    p_id_reservasi INT
```

```

)
RETURNS DECIMAL(12,2)
READS SQL DATA
BEGIN
    DECLARE v_total_kamar      DECIMAL(12,2);
    DECLARE v_total_tambahan DECIMAL(12,2);

    SELECT COALESCE(SUM(subtotal), 0) INTO v_total_kamar
    FROM detail_reservasi
    WHERE id_reservasi = p_id_reservasi;

    SELECT COALESCE(biaya_tambahan, 0) INTO v_total_tambahan
    FROM checkout
    WHERE id_reservasi = p_id_reservasi;

    RETURN v_total_kamar + v_total_tambahan;
END;

-- Contoh penggunaan:
SELECT fn_total_pendapatan_reservasi(1);
-- Output: 700000.00 (2 malam × Rp350.000

```

3. Function cek status pembayaran

Tujuan: Membandingkan total tagihan dengan total pembayaran lunas untuk menentukan status billing teks (BELUM BAYAR / KURANG BAYAR / LUNAS).
Memanfaatkan fn_total_pendapatan_reservasi untuk menghindari duplikasi logika.

```

CREATE FUNCTION fn_cek_status_pembayaran(
    p_id_reservasi INT
)
RETURNS VARCHAR(30)
READS SQL DATA
BEGIN
    -- Memanggil function lain (reusability)
    SET v_total_tagihan =
        fn_total_pendapatan_reservasi(p_id_reservasi);

    SELECT COALESCE(SUM(jumlah_bayar), 0) INTO v_total_bayar
    FROM pembayaran
    WHERE id_reservasi = p_id_reservasi
        AND status_pembayaran = 'Lunas';

    IF v_total_bayar = 0 THEN
        RETURN 'BELUM BAYAR';
    ELSEIF v_total_bayar < v_total_tagihan THEN
        RETURN 'KURANG BAYAR';
    ELSE
        RETURN 'LUNAS';
    END IF;
END;

-- Implementasi: laporan billing seluruh reservasi
SELECT
    id_reservasi,
    fn_hitung_durasi_malam(
        tanggal_checkin_rencana,
        tanggal_checkout_rencana) AS durasi_malam,
    fn_total_pendapatan_reservasi(
        id_reservasi) AS total_tagihan,

```

```
fn_cek_status_pembayaran(
    id_reservasi) AS status_keuangan
FROM reservasi;
```

Query implementasi di atas menggabungkan ketiga function dalam satu SELECT untuk menghasilkan laporan billing ringkas seluruh reservasi, contoh nyata composition of functions dalam SQL.

5.6 Ringkasan Objek Lanjutan

Jenis Objek	Nama	File	Status
VIEW	vw_detail_reservasi_tamu	04_view.sql	✓ Diimplementasikan
VIEW	vw_laporan_pembayaran	04_view.sql	✓ Diimplementasikan
VIEW	vw_billing_reservasi_summary	04_view.sql	✓ Diimplementasikan
VIEW	vw_status_kamar_opsional	04_view.sql	✓ Diimplementasikan
VIEW	vw_performa_staf_operasional	04_view.sql	✓ Diimplementasikan
STORED PROCEDURE	sp_hitung_invoice_lengkap	05_procedure.sql	✓ Diimplementasikan
STORED PROCEDURE	sp_proses_pembayaran_aman	05_procedure.sql	✓ Diimplementasikan
STORED PROCEDURE	sp_batal_reservasi_otomatis	05_procedure.sql	✓ Diimplementasikan
TRIGGER	trg_before_detail_reservasi_insert	06_trigger.sql	✓ Diimplementasikan
TRIGGER	trg_after_detail_reservasi_insert	06_trigger.sql	✓ Diimplementasikan
TRIGGER	trg_after_checkin_insert	06_trigger.sql	✓ Diimplementasikan
TRIGGER	trg_after_checkout_insert	06_trigger.sql	✓ Diimplementasikan
TRIGGER	trg_after_pembayaran_insert	06_trigger.sql	✓ Diimplementasikan
FUNCTION	fn_hitung_durasi_malam	07_function.sql	✓ Diimplementasikan

Jenis Objek	Nama	File	Status
FUNCTION	fn_total_pendapatan_reservasi	07_function.sql	✓ Diimplementasikan
FUNCTION	fn_cek_status_pembayaran	07_function.sql	✓ Diimplementasikan

BAB 6

PENUTUP

6.1 Kesimpulan

Project ini berhasil merancang dan mengimplementasikan Sistem Basis Data Relasional Reservasi Hotel secara menyeluruh menggunakan DBMS MySQL 8.0. Beberapa pencapaian utama yang dapat disimpulkan adalah sebagai berikut:

1. Sistem berhasil dimodelkan dengan 12 entitas/tabel yang saling terhubung melalui foreign key constraint, mencakup seluruh siklus bisnis hotel mulai dari reservasi, check-in, check-out, hingga pembayaran.
2. Proses normalisasi dilakukan secara sistematis dari Unnormalized Form (UNF) hingga Third Normal Form (3NF), mengeliminasi seluruh partial dependency dan transitive dependency sehingga menjamin konsistensi dan efisiensi penyimpanan data.
3. Implementasi DDL mencakup 12 jenis constraint (PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT, GENERATED, INDEX, ON UPDATE CASCADE, ON DELETE RESTRICT/CASCADE/SET NULL) yang memastikan integritas data di tingkat database.
4. Sebanyak 7 query SELECT dengan tingkat kompleksitas bertingkat berhasil dibuat, mulai dari query sederhana dengan JOIN 2 tabel, query dengan JOIN 6 tabel sekaligus, CTE (Common Table Expression), subquery, hingga agregasi multi-fungsi dengan GROUP BY dan HAVING.
5. Lima view diimplementasikan untuk menyederhanakan akses data bagi berbagai peran pengguna (resepsionis, kasir, housekeeping, manajemen).
6. Tiga stored procedure dirancang dengan prinsip ACID (menggunakan START TRANSACTION/COMMIT), validasi guard clause, dan reusability antar prosedur.
7. Lima trigger berhasil mengotomasi perubahan status kamar dan audit trail tanpa intervensi manual, termasuk perlindungan anti-double booking tingkat database menggunakan BEFORE INSERT trigger.
8. Tiga stored function dibuat dengan memanfaatkan konsep function composition (fn_cek_status_pembayaran memanggil fn_total_pendapatan_reservasi) untuk menghindari duplikasi logika bisnis.

6.2 Saran

Meskipun sistem telah berjalan sesuai spesifikasi, terdapat beberapa aspek yang dapat dikembangkan lebih lanjut:

1. Integrasi dengan lapisan aplikasi (front-end berbasis web atau mobile) agar sistem dapat digunakan secara interaktif oleh pegawai hotel tanpa perlu mengeksekusi SQL secara langsung.
2. Penambahan modul manajemen SDM (jadwal shift pegawai, penggajian) untuk melengkapi sistem operasional hotel secara menyeluruh.
3. Implementasi partisi tabel (table partitioning) pada tabel reservasi dan pembayaran untuk mengoptimalkan performa query ketika volume data sudah mencapai skala besar.
4. Penambahan fitur pencarian ketersediaan kamar berbasis rentang tanggal yang dapat diakses oleh tamu secara mandiri (self-service booking portal).
5. Penggunaan event scheduler MySQL untuk mengotomasi tugas periodik seperti pengiriman notifikasi reminder check-in kepada tamu atau pembatalan otomatis reservasi yang tidak terbayar dalam batas waktu tertentu.
6. Enkripsi kolom sensitif (no_identitas, email, no_telepon) menggunakan AES_ENCRYPT/AES_DECRYPT untuk kepatuhan terhadap regulasi perlindungan data pribadi.

LAMPIRAN

Link Github: <https://github.com/Ucoups/hotel-reservation-system>

Link AI: <https://chatgpt.com/share/6a2a5930-30d4-83ec-875c-870bb6f4ef1b>

<https://chatgpt.com/share/6a2a590b-c5e0-83ec-83d7-b147c03bdf4c>